دانشکده مهندسی برق

گزارش پروژه کارشناسی

در رشته مهندسی برق گرایش مخابرات

# شناسایی رتینوپاتی دیابتی با استفاده از یادگیری عمیق

نگارش:

امیررضا مسعودی

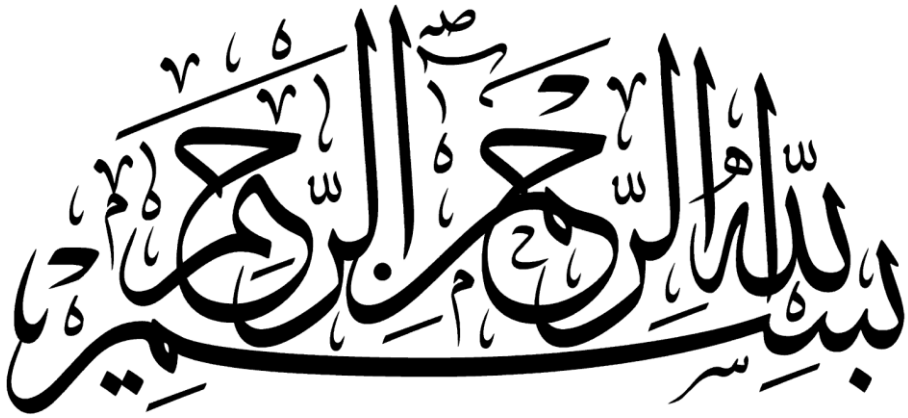
شماره دانشجویی:

۴۰۰۱۱۳۰۶۲

استاد راهنما:

دکتر حبیب اله دانیالی

شهریور ماه ۱۴۰۴


بسم الله الرحمن الرحيم

# چکیده

## شناسایی رتینوپاتی دیابتی با استفاده از یادگیری عمیق

**نگارش:**
**امیررضا مسعودی**

رتینوپاتی دیابتی یکی از بیماری‌های چشمی ناشی از دیابت است که بر اثر آسیب به عروق خونی شبکیه ایجاد می‌شود و در صورت عدم تشخیص و درمان به‌موقع می‌تواند منجر به نابینایی گردد. این بیماری از مهم‌ترین علل نابینایی در جهان به شمار می‌رود و تشخیص زودهنگام آن نقش تعیین‌کننده‌ای در پیشگیری از پیشرفت آسیب‌های چشمی دارد. روش‌های سنتی تشخیص معمولاً نیازمند معاینات تخصصی و زمان‌بر هستند؛ ازاین‌رو بهره‌گیری از روش‌های نوین مبتنی بر هوش مصنوعی و یادگیری عمیق می‌تواند دقت و سرعت فرایند تشخیص را به میزان قابل توجهی افزایش دهد. در این پروژه، با استفاده از شبکه‌های عصبی پیچشی (CNN) و داده‌های معتبر از پایگاه‌های APTOS ۲۰۱۹، APTOS ۲۰۱۵، IDRiD وMESSIDOR ، یک سامانه خودکار برای تشخیص رتینوپاتی دیابتی طراحی شده است که هدف آن تسهیل فرایند غربالگری بیماران می‌باشد. معماری این سامانه با الهام از یک مقاله منتخب طراحی شده و در آن از روش یادگیری انتقالی همراه با کدکننده EfficientNet-B1 بهره گرفته شده است. در نهایت، دو مدل توسعه یافته است: در مدل دودسته‌ای به دقت ٪۹۸.۳۶ و امتیاز QWK برابر با ۰.۹۶۷۲ دست یافته‌ایم و در مدل پنج‌دسته‌ای دقت ٪۸۴.۴۴ و امتیاز QWK برابر با ۰.۹۰۵۴ حاصل شده است.

# فهرست مطالب


**فصل اول: مقدمه** ۱

۱-۱- مقدمه ۲

۱-۲- بیان کلیات مسأله ۴

۱-۳- فصل‌های پایان‌نامه ۴

**فصل دوم: روش پیشنهادی برای حل مسئله** ۵

۲-۱- مقدمه ۶

۲-۲- مجموعه داده ها و معیار ارزیابی ۷

۲-۳- پیش پردازش و عملیات افزایش داده ۱۳

۲-۴- معماری شبکه ۱۶

۲-۴-۱- خانواده شبکه های کانولوشنی EfficientNet ۲۱

۲-۴-۲-بلوک MBConv ۲۳

۲-۵- فرآیند آموزش ۲۵

۲-۵-۱- مرحله پیش آموزش ۲۵

۲-۵-۲- مرحله آموزش اصلی ۲۹

۲-۵-۳- مرحله پسا آموزش ۳۳

۲-۵-۴- منظم سازی و مرحله تجمیع ۳۳

۲-۶- نتیجه گیری ۳۵

**فصل سوم: نتایج شبیه‌سازی یا پیاده‌سازی** ۳۶

۳-۱- مقدمه ۳۷

۳-۲- فرآیند آموزش ۳۷

۳-۲-۱- مرحله پیش آموزش ۳۹

۳-۲-۲- مرحله آموزش اصلی ۴۲

۳-۲-۳- نتایج نهایی ۴۴

۳-۳- ابزار ها و محیط های محاسباتی استفاده شده ۴۸

۳-۴- نتیجه گیری ۴۹

**فصل چهارم: جمع‌بندی و پیشنهادها** ۵۰

۴-۱- پرسش مهم ۵۱

۴-۲- پیشنهادها ۵۱

مراجع ۵۲

# فصل اول: مقدمه

## ۱-۱- مقدمه

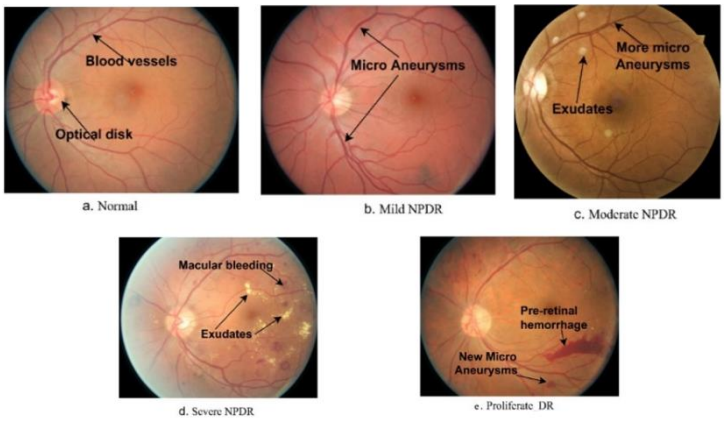
رتینوپاتی دیابتی [۱]یکی از خطرناک‌ترین عوارض بیماری دیابت است که در آن آسیب‌هایی به شبکیه وارد می‌شود و ممکن است منجر به نابینایی گردد. این بیماری به رگ‌های خونی موجود در بافت شبکیه آسیب می‌زند، باعث نشت مایع از آن‌ها شده و موجب اختلال در بینایی می‌شود. طبق آمارهای منتشرشده از ایالات متحده، بریتانیا و سنگاپور، در کنار بیماری‌هایی که به نابینایی می‌انجامند، مانند آب‌مروارید و گلوکوم، رتینوپاتی دیابتی یکی از شایع‌ترین اختلالات به شمار می‌رود.

رتینوپاتی دیابتی شامل ۴ مرحله می باشد:

۱- **رتینوپاتی غیرتکثیری خفیف**[۲]: ابتدایی ترین مرحله‌ی بیماری است که در آن تنها ریزآنوریسم‌ها[۳] ممکن است ظاهر شوند.
۲- **رتینوپاتی غیرتکثیری متوسط**[٤]: مرحله‌ای است که در آن با پیشرفت بیماری، رگ‌های خونی دچار پیچ‌خوردگی و تورم می‌شوند و توانایی آن‌ها در انتقال خون کاهش می‌یابد. در این مرحله ترشحات شبکیه ها به صورت اگزوداهای نرم یا لکه‌های پنبه‌ای و اگزوداهای سخت یا رسوبات لیپیدی قابل مشاهده هستند.
۳- **رتینوپاتی غیرتکثیری شدید**[٥]: منجر به کاهش خون‌رسانی به شبکیه می‌شود، زیرا انسداد تعداد بیشتری از رگ‌های خونی رخ می‌دهد. این وضعیت باعث می‌شود شبکیه سیگنال‌هایی برای رشد رگ‌های خونی جدید ارسال کند.
٤- **رتینوپاتی دیابتی تکثیری**[٦]: مرحله‌ی پیشرفته بیماری است که در آن، مواد رشد ترشح‌شده از سوی شبکیه باعث فعال شدن فرآیند تکثیر رگ‌های خونی جدید می‌شوند. این رگ‌های خونی تازه در امتداد سطح داخلی شبکیه و در برخی نواحی از زجاجیه رشد می‌کنند و ممکن است فضای داخلی چشم را پر کنند . در این مرحله خونریزی نیز اتفاق می افتد.

هر مرحله از رتینوپاتی دیابتی دارای ویژگی‌ها و خصوصیات خاص خود است، بنابراین ممکن است پزشکان برخی از آن‌ها را در نظر نگیرند و در نتیجه تشخیص نادرستی بدهند. این مسئله منجر به ایده‌ی ایجاد یک راه‌حل خودکار برای شناسایی رتینوپاتی دیابتی شده است.

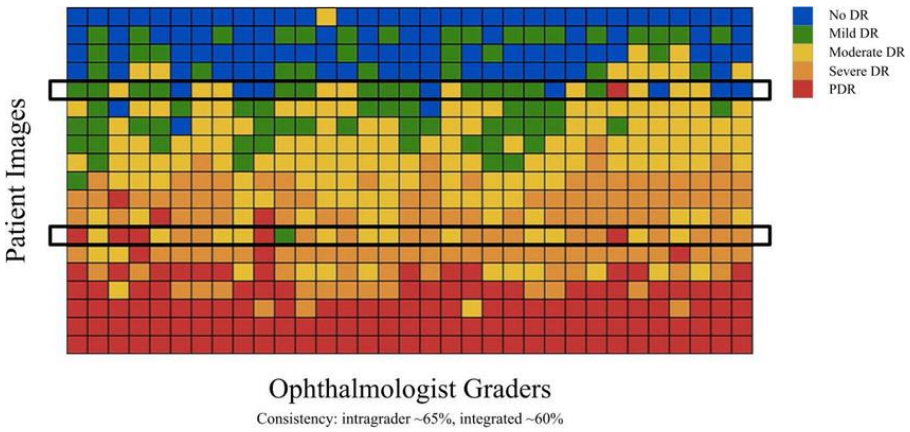
---
[۱] Diabetic Retinopathy
[۲] Mild Non-Proliferative Retinopathy
[۳] Microaneurysms
[٤] Moderate Non-Proliferative Retinopathy
[٥] Severe Non-Proliferative Retinopathy
[٦] Proliferative Diabetic Retinopathy




شکل ۱-۱ – تصاویر مربوط به مراحل بیماری رتینوپاتی دیابتی

دست‌کم ٪۵۶ از موارد جدید این بیماری می‌تواند با درمان به‌موقع و نظارت مناسب بر وضعیت چشم‌ها کاهش یابد [1]. با این حال، مرحله‌ی اولیه این بیماری هیچ نشانه هشداردهنده‌ای ندارد و همین موضوع، تشخیص زودهنگام آن را به یک چالش جدی تبدیل می‌کند.

علاوه بر این، طبق تحقیق شرکت گوگل، حتی پزشکان متخصص و آموزش‌دیده نیز گاهی نمی‌توانند به‌طور دستی مرحله بیماری را از روی تصاویر داخلی چشم [۱]به‌درستی تشخیص دهند [2]. در عین حال، در مواردی که ضایعات واضح باشند، اغلب پزشکان بر سر تشخیص با یکدیگر توافق دارند.




شکل ۲-۱ :گوگل نشان داد که تشخیص چشم‌پزشکان برای یک تصویر فوندوس مشابه با یکدیگر تفاوت دارد. (برای مشاهده بهتر، تصویر را به‌صورت رنگی ببینید.)

[۱] fundas

افزون بر این، روش‌های فعلی تشخیص کارایی چندانی ندارند، زیرا هم زمان‌بر هستند و هم نیاز به تعداد زیادی چشم‌پزشک برای بررسی مشکل بیمار دارند. این منابع اختلاف نظر منجر به تشخیص‌های نادرست و پایه‌های ناپایدار برای راه‌حل‌های خودکاری می‌شوند که قرار بود در مرحله‌ی پژوهش به کمک بیایند.

بنابراین، الگوریتم‌هایی برای تشخیص رتینوپاتی دیابتی به‌تدریج ظهور کردند. نخستین الگوریتم‌ها بر پایه‌ی روش‌های کلاسیک بینایی کامپیوتر و تعیین آستانه‌ها طراحی شده بودند. با این حال، در سال‌های اخیر، روش‌های یادگیری عمیق[1] برتری خود رادر وظایف طبقه‌بندی[۲] و تشخیص اشیا[۳] نسبت به سایر الگوریتم‌ها اثبات کرده‌اند.

به‌طور خاص، شبکه‌های عصبی پیچشی[٤] با موفقیت در بسیاری از حوزه‌های مرتبط و همچنین در تشخیص خودِ رتینوپاتی دیابتی به‌کار گرفته شده‌اند .

در فصل بعد به تشریح شبکه ای که برای تشخیص رتینوپاتی دیابتی به کار بردیم می پردازیم.

## ۲-۱- بیان کلیات مسئله

در این پروژه با الهام از یک مقاله منتخب، یک شبکه عصبی برای شناسایی بیماری رتینوپاتی دیابتی ارائه می دهیم. دو مدل را ارائه می دهیم. مدل اول یک مدل طبقه بند ۲ دسته ای و مدل دوم یک مدل طبقه بند ۵ دسته ای می باشد. در نهایت این مدل ها را با معیار های مناسب ارزیابی می کنیم.

## ۳-۱- فصل های پایان نامه

در فصل دوم به بررسی روش ارائه شده در مقاله اقتباسی و روش به کار گرفته شده در این پروژه و توضیحات مربوطه به قسمت های مختلف می پردازیم. در فصل سوم به ارزیابی عملکرد دو مدل ارائه شده می پردازیم.

---

[1] deep learning

[۲] classification

[۳] object detection

[٤] convolutional neural networks

# فصل دوم: روش پیشنهادی برای حل مسئله

## ۲-۱- مقدمه

در این فصل، ابتدا مجموعه‌داده‌ها و معیارهای ارزیابی مورد استفاده معرفی و بررسی می‌شوند. سپس فرایندهای پیش‌پردازش داده‌ها تشریح می‌گردند. در ادامه، معماری شبکه و کدکننده به‌کاررفته در آن توضیح داده می‌شود؛ زیرا در این پروژه از روش یادگیری انتقالی به‌منظور تسریع فرایند آموزش بهره گرفته شده است. در پایان، روند آموزش مدل شامل دو مرحلهٔ پیش‌آموزش و آموزش اصلی ارائه می‌شود که در آن جزئیات مراحل و توابع خطای به‌کاررفته شرح داده شده است. همچنین، به سایر جزئیات مرتبط از جمله منظم‌سازی و تجمیع نیز پرداخته خواهد شد.

روش کار در این پروژه اینطور بوده است که چندین مقاله بررسی شده و در نهایت یک مقاله انتخاب شده است [3] و روش آن مقاله به کار گرفته شده است. ضمنا هر دو هدف تشخیص رتینوپاتی دیابتی (به صورت وجود یا عدم وجود بیماری) و طبقه بندی بیماری به ۵ دسته دنبال شده است. عنوان مقاله "رویکرد یادگیری عمیق برای تشخیص رتینوپاتی دیابتی" [۱]می باشد.

در سال ۲۰۱۹، انجمن چشم‌پزشکی از راه دور آسیا-اقیانوسیه (اپتوس)[۲]و پلتفرم رقابتی یادگیری ماشین کگل[۳]، پژوهشگران حوزه یادگیری ماشین و یادگیری عمیق را به چالش کشیدند تا یک راه‌حل خودکار برای تشخیص رتینوپاتی دیابتی در پنج مرحله ارائه دهند (با استفاده از مجموعه‌داده تشخیص نابینایی)[۴]. در این مقاله، رویکرد یادگیری انتقالی [۵]و یک روش خودکار برای شناسایی مرحله‌ی رتینوپاتی دیابتی تنها از طریق یک عکس از فوندوس چشم انسان را پیشنهاد شده است. این رویکرد حتی از یک مجموعه‌داده کوچک و پرنویز نیز می‌تواند ویژگی‌های مفیدی بیاموزد و به‌عنوان یک روش غربالگری مراحل بیماری در سیستم‌های خودکار مورد استفاده قرار گیرد. همچنین، این روش توانست در رقابت تشخیص نابینایی اپتوس ۲۰۱۹ رتبه‌ی ۵۴ از میان ۲۹۴۳ روش مختلف را کسب کند و به امتیاز کاپای وزنی درجه دوم [۶]برابر با ۰.۹۲۵۴۶ دست یابد.

---

[۱] Deep Learning Approach to Diabetic Retinopathy Detection

[۲] APTOS

[۳] Kaggle

[۴] APTOS 2019 Blindness Detection Dataset

[۵] transfer learning

[۶] Quadratic Weighted Kappa

## ۲-۲- مجموعه داده ها و معیار ارزیابی

داده‌های تصویری مورد استفاده در این تحقیق از چندین مجموعه‌داده مختلف گردآوری شده‌اند. ما برای پیش‌آموزش شبکه‌های عصبی پیچشی خود، از یک مجموعه‌داده‌ی متعلق به رقابت کگل سال ۲۰۱۵ [۱] استفاده کردیم. این مجموعه‌داده بزرگ‌ترین مجموعه‌ی در دسترس عمومی است و شامل ۳۵۱۲۶ تصویر فوندوس از چشم چپ و راست شهروندان آمریکایی می‌باشد که با برچسب مراحل مختلف رتینوپاتی دیابتی مشخص شده‌اند:

- بدون رتینوپاتی دیابتی (برچسب ۰)
- رتینوپاتی دیابتی خفیف (برچسب ۱)
- رتینوپاتی دیابتی متوسط (برچسب ۲)
- رتینوپاتی دیابتی شدید (برچسب ۳)
- رتینوپاتی دیابتی تکثیری (برچسب ۴)

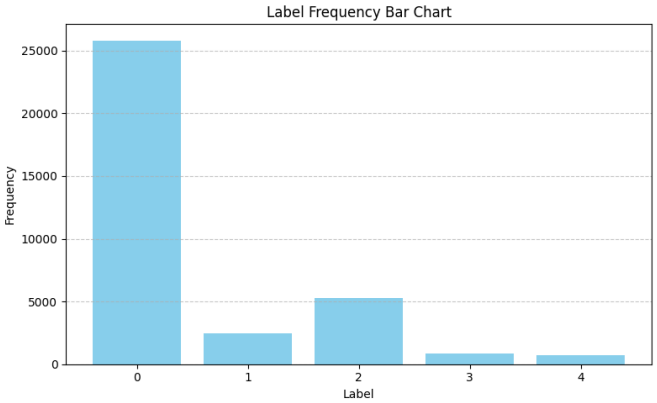
علاوه بر آن، از مجموعه‌داده‌های کوچک‌تری نیز استفاده کردیم: مجموعه‌داده تصویری رتینوپاتی دیابتی هند [۲](ایدی ارایدی) که شامل ۴۵۵ تصویر فوندوس است و مجموعه‌داده مسیدور[3] که از آن ۱۷۴۴ تصویر فوندوس مورد استفاده قرار گرفت. مجموعه داده های اپتوس ۲۰۱۹ که هدف این پروژه می باشد، دارای ۳۶۶۲ تصویر فوندوس است که ۲۰ درصد آن یعنی حدود ۷۳۳ عکس به عنوان دیتاست آزمایش [٤]انتخاب شده است که این دیتاست در فرآیند آموزش توسط مدل مشاهده نشده است. توزیع داده ها در مجموعه داده ها در نمودار های مربوطه نشان داده شده است.

[۱] Kaggle Diabetic Retinopathy Detection Challenge 2015

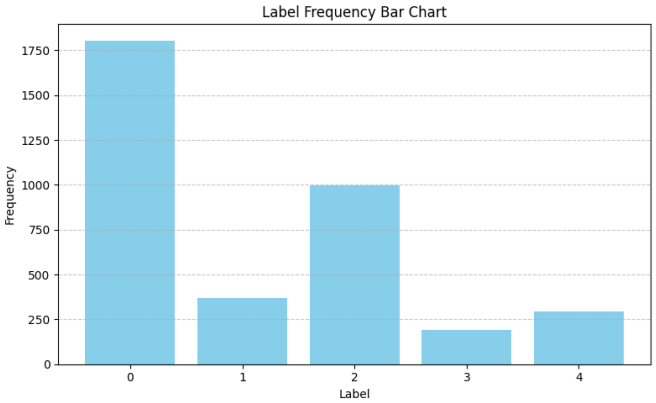
[۲] IDRiD

[3] MESSIDOR

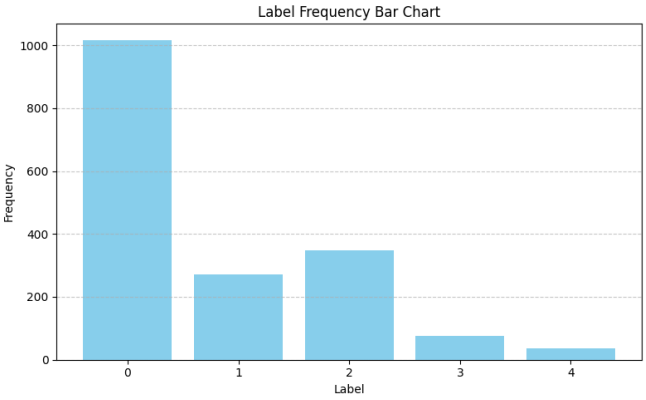
[٤] test dataset



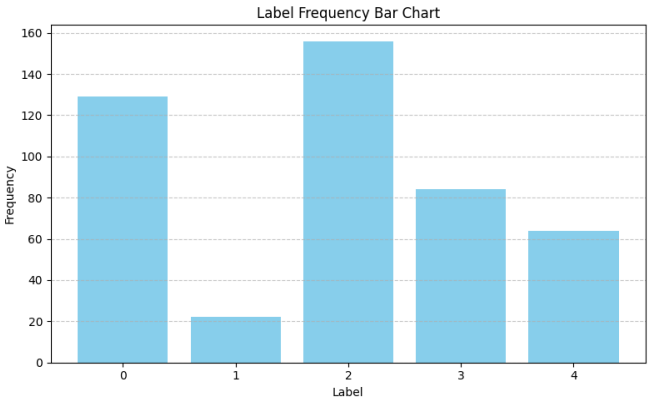

شکل ۲-۱ توزیع داده ها در APTOS ۲۰۱۵




شکل ۲-۲ توزیع داده ها در APTOS ۲۰۱۹




شکل ۲-۳ توزیع داده ها در MESSIDOR




شکل ۲-۴ توزیع داده ها در IDRiD

در این پروژه، از امتیاز کوهن کاپای وزنی درجه دوم [۱] به‌عنوان معیار اصلی ارزیابی استفاده کردیم. امتیاز کاپا میزان توافق بین دو ارزیاب را اندازه‌گیری می‌کند. در این پروژه، کاپای وزنی درجه دوم بین نمراتی که توسط انسان (پزشک) اختصاص داده شده و نمرات پیش‌بینی‌شده توسط مدل محاسبه می‌شود. این معیار عددی بین 1- (عدم توافق کامل بین ارزیاب‌ها) تا ۱ (توافق کامل) می باشد.در ادامه مثالی جهت توضیح این معیا آورده شده است.

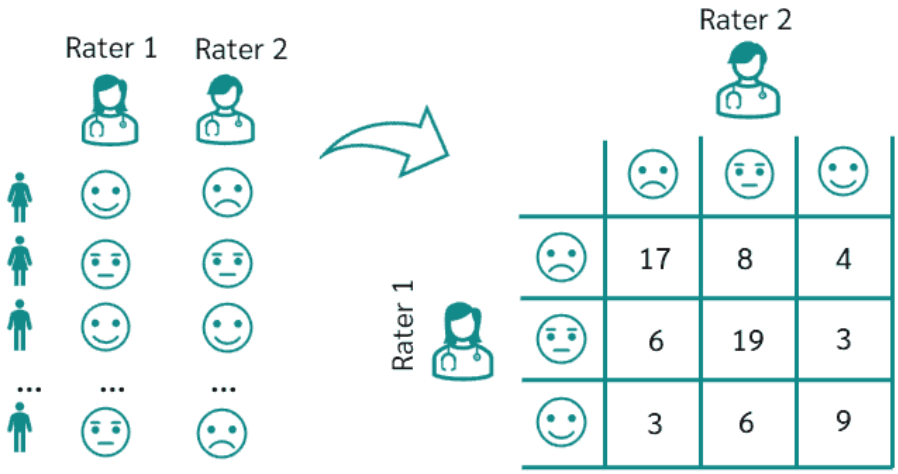
## امتیاز کوهن کاپای وزنی درجه دوم

فرض کنید دو پزشک جداگانه میزان رضایت خود را از موفقیت درمان بیمارانشان ارزیابی کرده‌اند. پاسخ این پزشکان می‌تواند یکی از سه گزینه باشد : ناراضی، خنثی و راضی. در اینجا ۷۵ نفر بیمار داریم.

[۱] Quadratic Weighted Cohen's Kappa

اکنون می‌خواهیم بدانیم تا چه اندازه توافق بین دو پزشک وجود دارد. از آن‌جایی که با یک متغیر ترتیبی سر و کار داریم که ترتیب آن‌ها به صورت *ناراضی* < *خنثی* < *راضی* است، برای اندازه‌گیری میزان توافق از کاپای وزنی کوهن استفاده می‌کنیم.

گام اول این است که جدولی از فراوانی پاسخ‌ها تهیه کنیم. در این جدول، یکی از ارزیاب‌ها (پزشکان) را روی محور افقی و دیگری را روی محور عمودی قرار می‌دهیم. این جدول نشان می‌دهد که هر یک از دو پزشک، در مورد موفقیت درمان هر بیمار، چه نظری داشته‌اند (ناراضی، خنثی یا راضی).




با توجه به جدول، برای مثال هر دو پزشک از درمان ۱۷ نفر از بیماران ناراضی بوده اند و از درمان ۹ نفر راضی بوده اند.

امتیاز کوهن کاپای وزنی از فرمول زیر محسابه می شود.

$$\kappa_w = 1 - \frac{\sum w_{ij} \cdot f_{o\,ij}}{\sum w_{ij} \cdot f_{e\,ij}}$$

Weighting factors — Observe frequencies — Expected frequencies

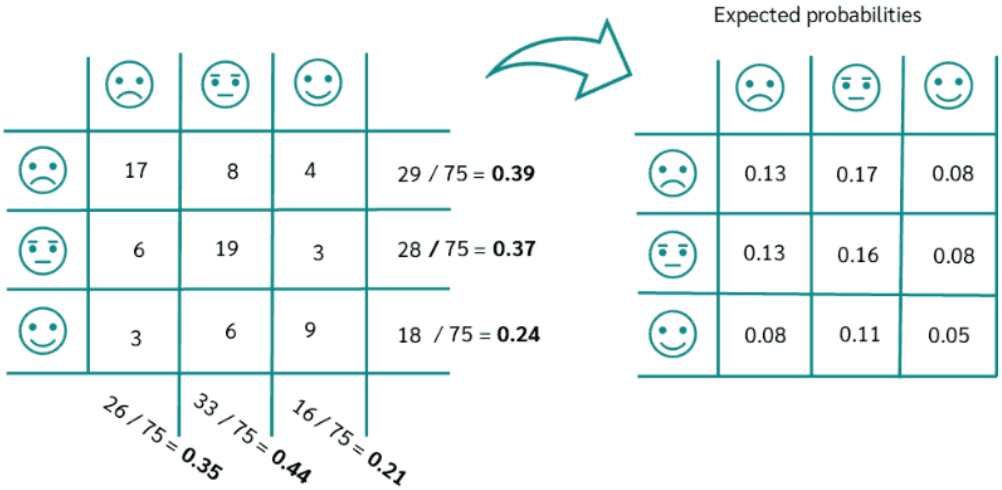
در اینجا، $w_{ij}$ نمایانگر ضریب‌های وزن‌دهی [۱] است، $f_{o\,ij}$ نمایانگر فراوانی های مشاهده شده [۲] است و $f_{e\,ij}$ نمایانگر فراوانی های مورد انتظار [۳] می باشد.

[۱] weighting factors

[۲] observed

[۳] expected

برای محاسبه فراوانی مورد انتظار، ابتدا مجموع سطرها و ستون‌ها را محاسبه می‌کنیم. یعنی تمام مقادیر هر سطر و هر ستون را جمع می‌زنیم.
برای مثال، در سطر اول مجموع برابر با ۲۹ می‌شود که از جمع ۱۷ + ۸ + ۵ به دست آمده است. سپس این عدد را بر ۷۵ که تعداد کل نمونه‌ها است تقسیم می‌کنیم.

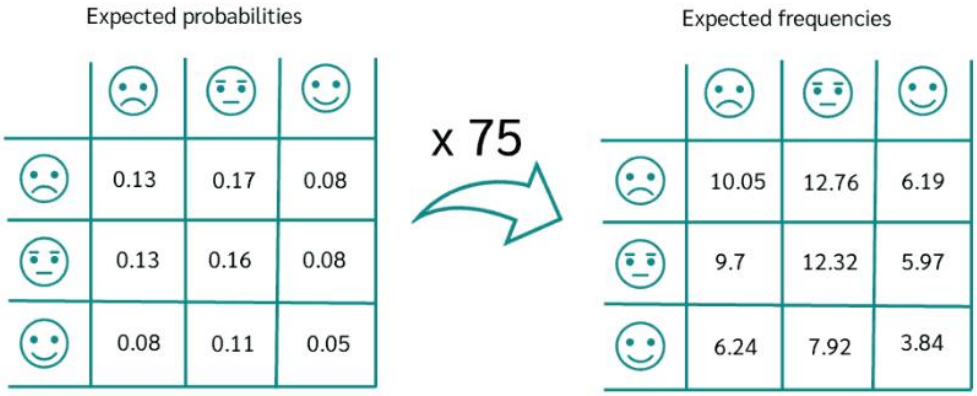



اکنون می‌توانیم احتمال مورد انتظار برای هر خانه را با ضرب احتمال سطری در احتمال ستونی محاسبه کنیم.

برای مثال:

- در اولین خانه، حاصل ضرب ۰.۳۵ در ۰.۳۹ برابر است با ۰.۱۳
- در خانه دوم، حاصل ضرب ۰.۴۴ در ۰.۳۹ برابر است با ۰.۱۷

حال اگر هر یک از این احتمالات را در ۷۵ (تعداد کل نمونه‌ها) ضرب کنیم، فراوانی‌های مورد انتظار برای هر خانه به دست می‌آیند. مثلا احتمال مورد انتظار برای اینکه هر دو پزشک امتیاز راضی را دهند، ۰.۰۵ یا ٪۵ می باشد.



کاپای کوهن بررسی می‌کند که آیا دو ارزیاب (مثلاً دو پزشک) یک چیز را به‌درستی اندازه‌گیری کرده‌اند یا نه، اما درجه اختلاف بین آن‌ها را در نظر نمی‌گیرد. اگر بخواهیم شدت اختلاف بین پاسخ‌ها را نیز در محاسبه لحاظ کنیم، باید از کاپای وزنی کوهن (Weighted Cohen's Kappa) استفاده کنیم. به این علت به مقادیر وزن ها نیز نیاز پیدا می کنیم.

اگر از هیچ‌گونه وزن‌دهی استفاده نکنیم، ماتریس وزن‌دهی ما تنها شامل عددهای صفر و یک در قطر و بیرون از آن خواهد بود.

- اگر هر دو ارزیاب پاسخ یکسانی داده باشند (یعنی بر سر یک گزینه توافق داشته باشند)، مقدار صفر در آن خانه (قطر اصلی ماتریس) قرار می‌گیرد.
- اگر پاسخ آن‌ها متفاوت باشد، مقدار ۱ در آن خانه قرار می‌گیرد.

در این حالت، میزان اختلاف بین پاسخ‌ها (مثلاً تفاوت بین "ناراضی" و "راضی" نسبت به تفاوت بین "ناراضی" و "خنثی") هیچ اهمیتی ندارد. فقط مهم است که آیا پاسخ‌ها متفاوت هستند یا نه، و در صورت تفاوت همگی به اندازه ۱ وزن دهی می شوند.

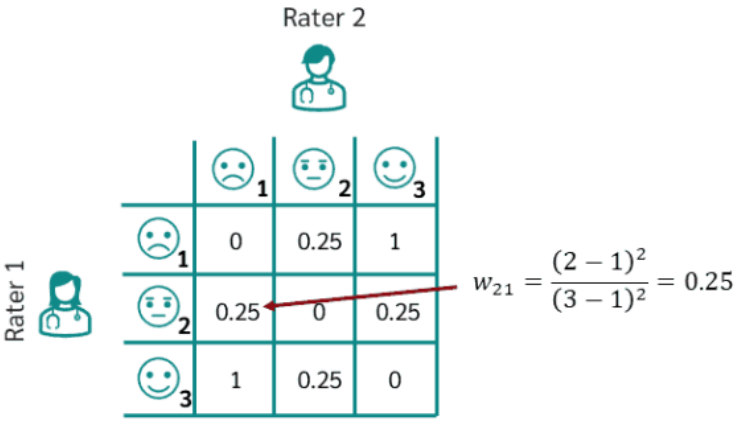
برای وزن دهی خطی، ماتریس وزن‌دهی خطی را می‌توان با استفاده از فرمول زیر محاسبه کرد k تعداد کلاس ها می باشد که در اینجا ۳ است.

$$w_{ij} = \frac{|i-j|}{k-1}$$

اگر به‌جای وزن‌دهی خطی از وزن‌دهی درجه‌دوم استفاده کنیم، فاصله‌ها به توان دو می‌رسند. به این ترتیب، امتیازهایی که با هم فاصله‌ی بیشتری دارند، نسبت به امتیازهای نزدیک‌تر، وزن بسیار بیشتری دریافت می‌کنند (یعنی اختلاف آن‌ها شدیدتر در نظر گرفته می‌شود) در مقایسه با حالت خطی.

ماتریس وزن‌دهی در این حالت با استفاده از فرمول زیر به دست می‌آید و بدین ترتیب ماتریس وزن های مسئله نیز به دست می آید:

$$w_{ij} = \frac{(i-j)^2}{(k-1)^2}$$



در نهایت می توانیم امتیاز کوهن کاپای وزنی درجه دوم را به دست آوریم که مقدار ۰.۳۹۶ می شود.

Weighting matrix

| | | |
|---|---|---|
| 0 | 0,5 | 1 |
| 0,5 | 0 | 0,5 |
| 1 | 0,5 | 0 |

Observed frequency

| | | |
|---|---|---|
| 17 | 8 | 4 |
| 6 | 19 | 3 |
| 3 | 6 | 9 |

$$\kappa_w = 1 - \frac{\sum w_{ij} \cdot p_{ij}}{\sum w_{ij} \cdot e_{ij}} = 1 - \frac{0 \cdot 17 + 0.5 \cdot 8 + \cdots + 0 \cdot 9}{0 \cdot 10.05 + 0.5 \cdot 12.76 + \cdots + 0 \cdot 3.84} = 1 - \frac{18.5}{30.6} = 0.396$$

| | | |
|---|---|---|
| 0 | 0.5 | 1 |
| 0.5 | 0 | 0.5 |
| 1 | 0.5 | 0 |

Weighting matrix

| | | |
|---|---|---|
| 10.05 | 12.76 | 6.19 |
| 9.7 | 12.32 | 5.97 |
| 6.24 | 7.92 | 3.84 |

Expected frequency

این امتیاز چالش هایی را به خود به همراه دارد. برای مثال اگر تعداد کلاس ها یعنی k زیاد باشد، مقدار این امتیاز بیشتر می شود و این می تواند گمراه کننده باشد. اما در این پروژه معیار مناسبی می باشد. همراه با امتیاز کاپا ، ما مقدارهای امتیاز ماکرو F1- [۱]، دقت[۲]، حساسیت [۳] و ویژگی [۴]را نیز بر روی مجموعه‌ی نگه‌داشته‌شده‌ای شامل ۷۳۳ تصویر از داده‌های آموزشی اپتوس ۲۰۱۹ محاسبه کردیم. این مجموعه به عنوان آزمایش در نظر گرفته شده و در فرآیند آموزش مدل استفاده نشده است.

[۱] macro F1-score

[۲] accuracy

[۳] sensitivity

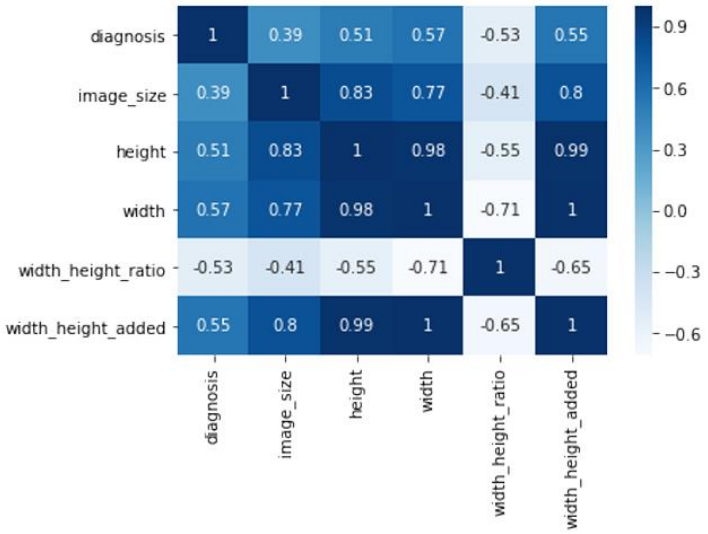
[۴] specificity

## روش کار:

مسئله تشخیص رتینوپاتی دیابتی را می‌توان از چندین زاویه بررسی کرد: به‌عنوان یک مسئله طبقه‌بندی، به‌عنوان یک مسئله رگرسیون، و به‌عنوان یک مسئله رگرسیون ترتیبی. این امر به این دلیل ممکن است که مراحل بیماری به‌صورت پی‌درپی رخ می‌دهند.

## ۳-۲- پیش پردازش و عملیات افزایش داده

آموزش و اعتبارسنجی مدل با نسخه‌های پیش‌پردازش‌شده تصاویر اصلی انجام شد. پیش‌پردازش شامل برش تصویر و سپس تغییر اندازه بود. به دلیل نحوه گردآوری مجموعه‌داده اپتوس ۲۰۱۹، همبستگی‌های کاذبی بین مرحله بیماری و چندین فرامتن (ویژگی‌های فراداده‌ای) تصویر وجود دارد؛ مانند وضوح تصویر، نوع برش، میزان بزرگ‌نمایی یا روشنایی کلی. ماتریس همبستگی در شکل ۱-۵-۲ نشان داده شده است. برای اینکه شبکه عصبی کانولوشنی روی این ویژگی‌ها بیش‌برازش [۱] نکند و برای کاهش همبستگی بین محتوای تصویر و فرامتن‌های آن، از مقدار زیاد عملیات افزایش داده [۲] استفاده کردیم.




شکل ۵-۲. ماتریس همبستگی تصاویر و ویژگی های کاذب

[۱] overfit

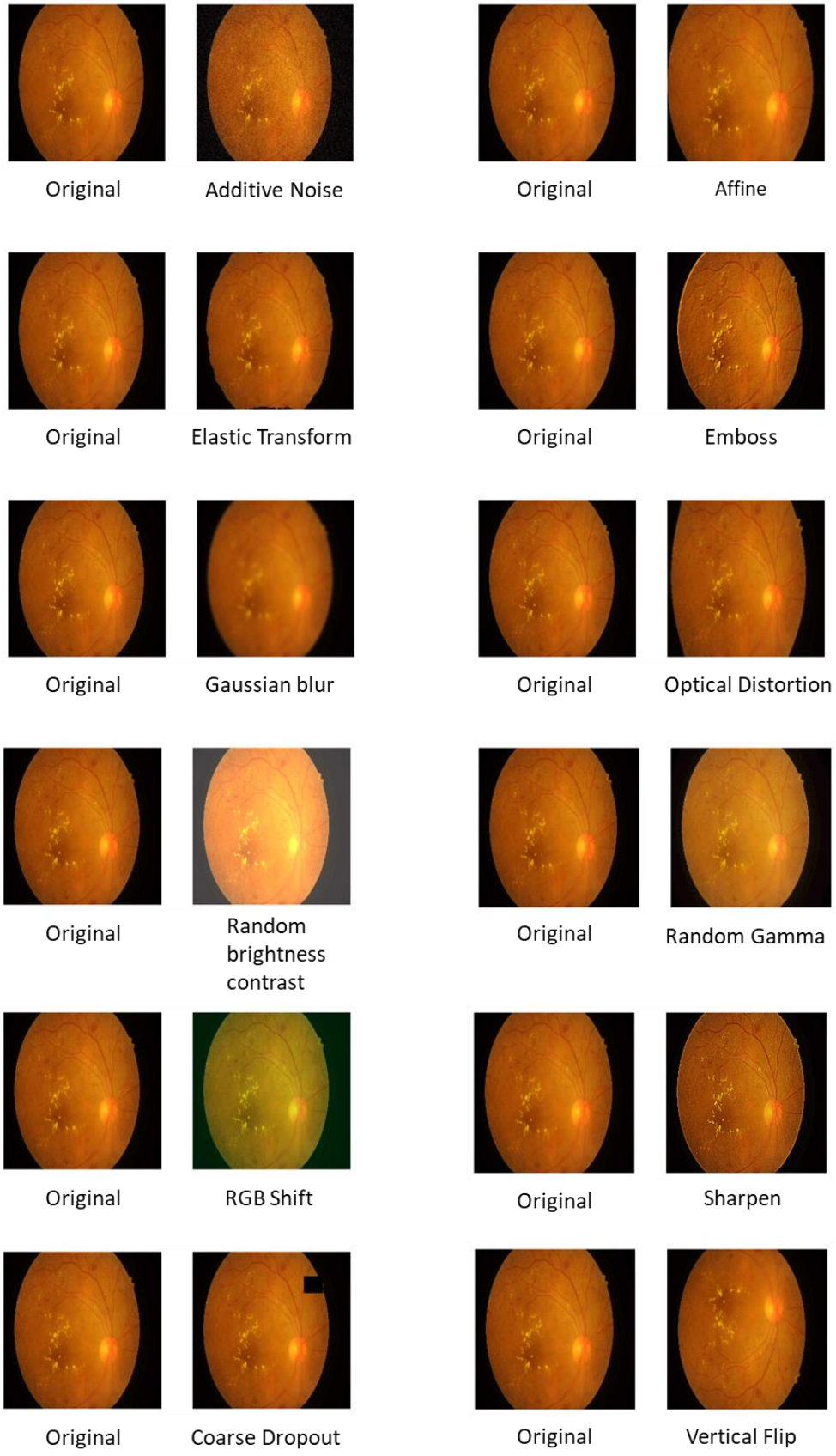
[۲] data augmentation

ما از افزایش داده‌های آنلاین [۱] استفاده کردیم؛ به این صورت که قبل از وارد کردن تصویر آموزشی به شبکه عصبی کانولوشنی، حداقل یک روش افزایش داده روی آن اعمال می‌شد، این یعنی از چند تغییر ممکن روی عکس ها، حداقل یک تغییر انتخاب شده و اعمال می شد.

ما در کتابخانه Albumentations [4] ، از افزایش‌های زیر استفاده کردیم: اعوجاج نوری [۲]، اعوجاج شبکه‌ای [۳]، تبدیل افاین قطعه‌ای [۴]، وارون‌سازی افقی [۵]، وارون‌سازی عمودی [۶]، چرخش تصادفی [۷]، جابه‌جایی تصادفی [۸]، تغییر مقیاس تصادفی [۹]، تغییر جابه‌جایی مقادیر RGB ، روشنایی و کنتراست تصادفی [۱۰]، نویز گاوسی افزوده [۱۱]، محو کردن [۱۲]، تیز کردن تصویر [۱۳]، برجسته‌سازی [۱۴]، گامای تصادفی [۱۵] و بریدن [۱۶].

شکل ۶-۲ نمونه هایی از عملیات افزایش داده انجام شده را نشان می دهد.

[۱] online augmentations
[۲] optical distortion
[۳] grid distortion
[۴] piecewise affine transform
[۵] horizontal flip
[۶] vertical flip
[۷] random rotation
[۸] random shift
[۹] random scale
[۱۰] random brightness and contrast
[۱۱] additive Gaussian noise
[۱۲] blur
[۱۳] sharpening
[۱۴] embossing
[۱۵] random gamma
[۱۶] cutout




شکل ۶-۲ – انواع عملیات افزایش داده، بعضی پارامتر ها در فرآیند آموزش مدل تغییر یافته اند و از شدت تغییر عکس کاسته شده است.

## ۲-۴- معماری شبکه

هدف ما این است که هر تصویر فوندوس را به‌طور دقیق طبقه‌بندی کنیم. ما شبکه‌های عصبی خود را با استفاده از معماری متداول شبکه عصبی کانولوشنی عمیق می‌سازیم که شامل یک استخراج‌کننده ویژگی [۱] و یک بخش رمزگشا [2] کوچک‌تر برای یک وظیفه خاص است. با این حال، آموزش بخش کدکننده [۳] از ابتدا دشوار است، به‌ویژه با توجه به مقدار کم داده‌های آموزشی. به همین دلیل، ما از شبکه‌های عصبی کانولوشنی آموزش‌دیده با مجموعه داده Imagenet به‌عنوان مقداردهی اولیه برای رمزگذار استفاده می‌کنیم.

یادگیری انتقالی یک روش مؤثر در یادگیری ماشین است که اجازه می‌دهد دانش استخراج‌شده توسط یک مدل از پیش آموزش‌دیده روی مجموعه‌داده‌های بزرگ و متنوع، برای حل یک وظیفه جدید با داده محدود و زمان آموزش کمتر به‌کار گرفته شود. این رویکرد به‌ویژه در تصویربرداری پزشکی و پروژه‌هایی مانند تشخیص رتینوپاتی دیابتی کاربردی است، چرا که جمع‌آوری و برچسب‌گذاری داده‌های پزشکی دشوار و هزینه‌بر است. شبکه‌های عصبی کانولوشنی به طور سلسله‌مراتبی ویژگی‌های تصاویر را یاد می‌گیرند؛ لایه‌های اولیه الگوهای ساده مانند خطوط و لبه‌ها را تشخیص می‌دهند، لایه‌های میانی شکل‌های پیچیده‌تر و بافت‌ها را استخراج می‌کنند و لایه‌های بالایی ویژگی‌های سطح بالا و معنادار تصویر را شناسایی می‌کنند.

همچنین استفاده از وزن‌های از پیش آموزش‌دیده باعث همگرایی سریع‌تر و کاهش خطر بیش‌برازش می‌شود و امکان آموزش تنها لایه‌های پایانی [٤] را فراهم می‌کند که هزینه محاسباتی را به‌طور چشمگیری کاهش می‌دهد. در مقاله اقتباسی از مدل های خانواده EfficientNet و ResNet استفاده شده است. ما در این پروژه از مدل EfficientNet استفاده کردیم که جلوتر به شرح جزییات آن خواهیم پرداخت.

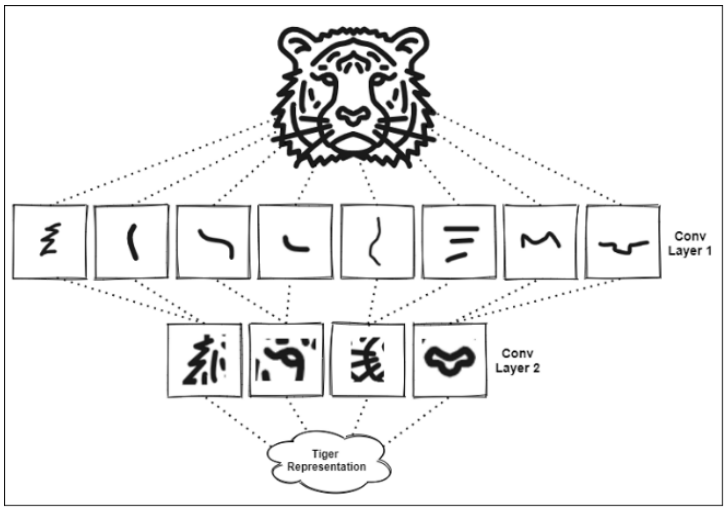
---

[۱] feature extractor

[2] decoder

[۳] encoder

[٤] fine-tuning




شکل ۷-۲. یادگیری شبکه های CNN – لایه اولیه شکل های ساده تر را تشخیص می دهند و هر چه جلوتر می رویم، لایه ها شکل های پیچیده تری را تشخیص می دهند. با یادگیری انتقالی، می توان فقط لایه های آخر را آموزش داد زیرا که مدل شکل های ساده تر با دیتاستی مانند Imagenet را فرا گرفته است.

در مقاله ای که مورد اقتباس قرار گرفته، رویکرد یادگیری چندوظیفه‌ای [۱]را برای شناسایی رتینوپاتی دیابتی پیشنهاد شده است. در این روش از سه رمزگشا استفاده می شود که هرکدام بر اساس ویژگی‌های استخراج‌شده توسط CNN پایه آموزش داده می‌شوند تا وظیفه خود را انجام دهند:

- سر طبقه‌بندی[2]
- سر رگرسیون[3]
- سر رگرسیون ترتیبی[4]

اینجا، سر طبقه بندی یک بردار وان-هات [۵]تولید می‌کند که در آن وجود هر مرحله با عدد ۱ نشان داده می‌شود. برای مثال اگر عکسی مربوط به کلاس ۴ (شدید ترین مرحله بیماری) باشد، بردار وان-هات آن به صورت زیر خواهد بود.

$$[0, 0, 0, 0, 1]$$

[1] multi-task learning

[2] classification head

[3] regression head

[4] ordinal regression head

[5] one-hot encoded

سر رگرسیون یک عدد حقیقی در بازه [0,4.5) خروجی می‌دهد که سپس گرد می‌شود تا یک عدد صحیح به دست آید که مرحله بیماری را نشان می‌دهد.
برای بخش رگرسیون ترتیبی، [1]از روشی که در [5] توضیح داده شده استفاده شده است. به طور خلاصه، اگر یک داده در دسته k قرار بگیرد، به طور خودکار در همه دسته‌های 0 تا k-1 نیز قرار می‌گیرد. در این پروژه k برابر با ۵ است چون ۵ دسته داریم. بنابراین این بخش هدف دارد که تمام دسته‌ها تا مقدار هدف را پیش‌بینی کند. در روش رگرسیون خطی مدل متوجه می شود که مراحل بیماری به تدریج پیشروی می کنند و این باعث بهبود یادگیری در فرآیند آموزش می شود. برای مثال اگر عکسی مربوط به کلاس ۲ (متوسط) باشد، بردار وات-هات آن به صورت زیر خواهد بود.

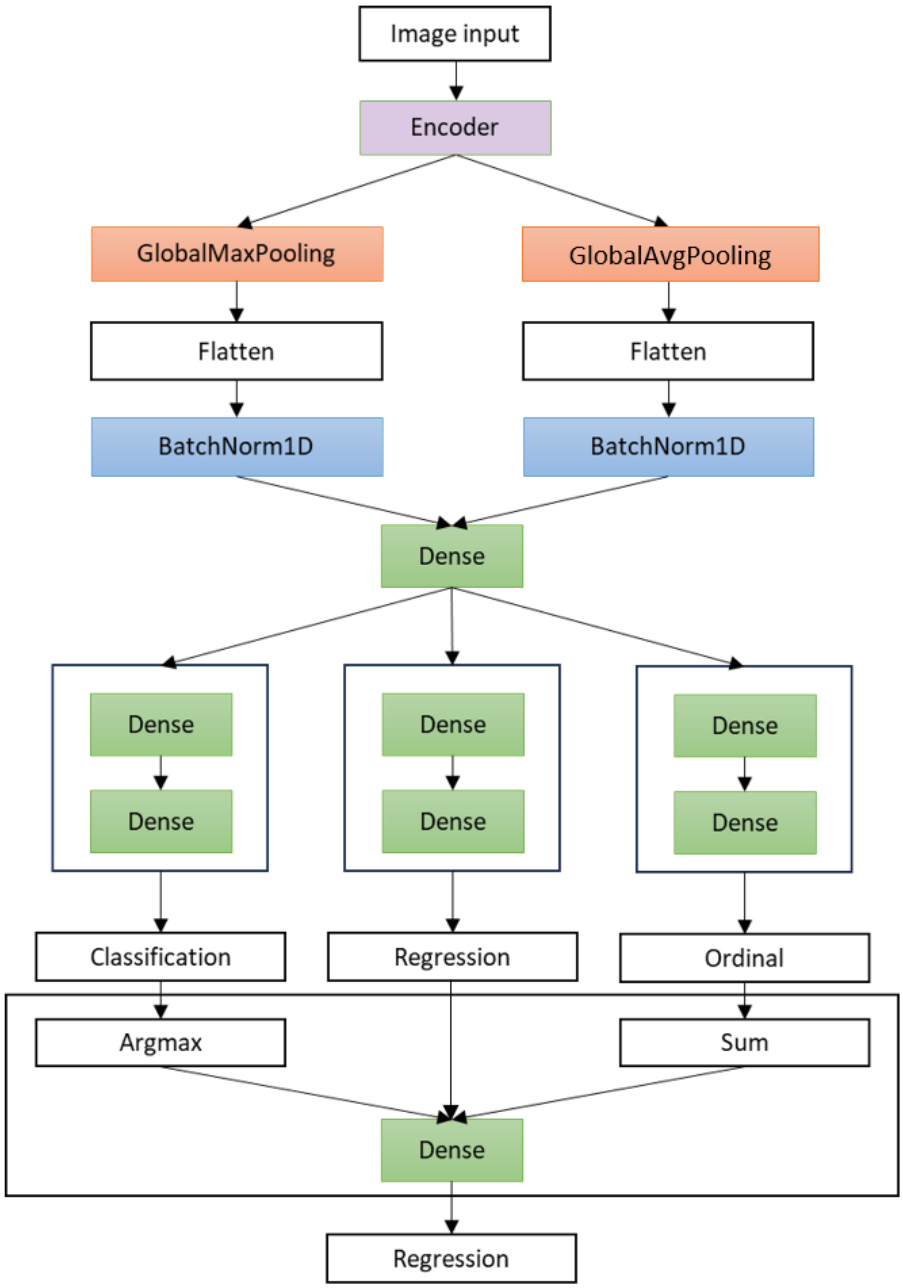
$$[1, 1, 0, 0]$$

پیش‌بینی نهایی با برازش یک مدل رگرسیون خطی روی خروجی‌های هر سه بخش به دست می‌آید. ساختار شبکه عصبی در شکل ۸-۲ نشان داده شده است. تمام سر ها و استخراج‌کننده ویژگی را به‌طور مشترک آموزش داده می شوند تا زمان آموزش کاهش یابد. مدل رگرسیون خطی را تا مرحله پس‌آموزش فریز (غیرفعال) نگه داشته می شود.

## روش در این پروژه

در این پروژه این نسخه ای ساده تر از این روش انتخاب شده است. به جای ترکیب ۳ سر، فقط یک سر انتخاب کردیم. این سر را یک بار طبقه بندی و بار دیگر رگرسیون ترتیبی انتخاب کردیم. بدین ترتیب دیگر نیازی به یک برازش نهایی که از ترکیب ۳ سر به دست می آید نیازی نخواهد بود (مرحله پسا آموزش). ساختار شبکه عصبی در شکل ۹-۲ نشان داده شده است.

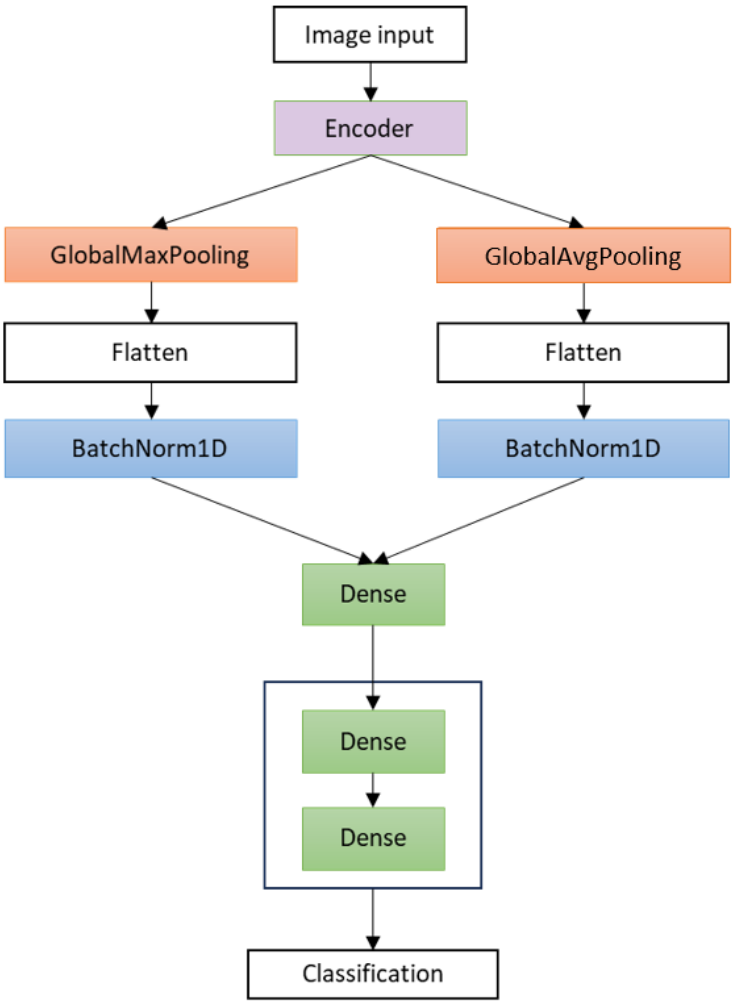
[1] ordinal regression head




شکل ۸-۲. معماری استفاده شده در مقاله اقتباسی

ما علاوه بر این که عکس ها را در ۵ دسته دسته بندی کنیم، نوع دیگر دسته بندی به صورت باینری را نیز به کار بردیم. یعنی عکس ها به دو دسته ۰ و ۱ تقسیم بندی می شوند که ۰ به معنای عدم وجود بیماری و ۱ به معنای وجود بیماری می باشد.

در نتایج شبیه سازی نشان خواهیم داد که معیار های ارزیابی در دسته بندی باینری بهبود زیادی دارند.




شکل ۹-۲ . معماری استفاده شده در این پروژه

## استخراج کننده ویژگی

در مقاله اقتباسی، از مدل های از پیش آموزش داده شده شامل EfficientNet-B4، EfficientNet-B5 ، (512*512) SE-ResNeXt50 و (380*380) SE-ResNeXt50 به عنوان استخراج کننده ویژگی استفاده شده است. ما در این پروژه برای کاهش تعداد پارامتر های قابل آموزش و کاهش حجم محاسبات، از EfficientNet-B1 استفاده کردیم که حدود ۶،۸۵۰،۲۷۳ پارامتر قابل آموزش دارد (در مقابل با تعداد ۱۹،۳۴۱،۶۱۶ پارامتر قابل آموزش برای EfficientNet-B4).
در ادامه به توضیح بیشتر درباره خانواده EfficientNet خواهیم داد.

## ۱-۴-۲- خانواده شبکه های کانولوشنی EfficientNet

EfficientNet خانواده‌ای از معماری‌های شبکه‌های عصبی پیچشی است که در سال ۲۰۱۹ توسط Google AI معرفی شد [12] . هدف اصلی EfficientNet دستیابی به بالاترین دقت در وظایف طبقه‌بندی تصویر در حالی است که کارایی محاسباتی نیز حفظ شود. برخلاف معماری‌های قبلی که به‌طور دلخواه تنها در عمق، عرض یا وضوح ورودی گسترش می‌یافتند، EfficientNet روشی به نام مقیاس‌بندی ترکیبی [۱]معرفی می‌کند که هر سه بعد را به صورت متعادل و اصولی مقیاس‌بندی می‌کند. این رویکرد باعث شده است که مدل‌های EfficientNet با تعداد پارامتر کمتر و هزینه محاسباتی پایین‌تر نسبت به مدل‌هایی مانند ResNet ، DenseNet یا Inception عملکرد بهتری داشته باشند.

استراتژی‌های مقیاس‌بندی سنتی در شبکه‌های کانولوشنی معمولاً یکی از ابعاد زیر را افزایش می‌دهند:

- مقیاس‌بندی عمق[۲]: افزودن لایه‌های بیشتر برای شناسایی الگوهای پیچیده‌تر (مانند ResNet-152)
- مقیاس‌بندی عرض[۳]: افزایش تعداد کانال‌ها در هر لایه برای ایجاد نقشه‌های ویژگی بیشتر (مانند wide ResNet)
- مقیاس‌بندی وضوح[٤]: افزایش اندازه ورودی تصویر برای دریافت جزئیات بیشتر.

با این حال، مقیاس‌بندی تنها یک بُعد اغلب به نتایج بهینه منجر نمی‌شود و یا هزینه محاسباتی بسیار بالایی دارد. EfficientNet با مقیاس‌بندی همزمان عمق، عرض و وضوح به صورت متعادل این مشکل را برطرف می‌کند.

نوآوری اصلی EfficientNet استفاده از فرمول مقیاس‌بندی ترکیبی است:

$$\text{عمق} = \alpha^{\phi}, \qquad \text{عرض} = \beta^{\phi}, \qquad \text{وضوح} = \gamma^{\phi}$$

با قید زیر:

$$\gamma^2 * \beta^2 * \alpha \approx 2$$

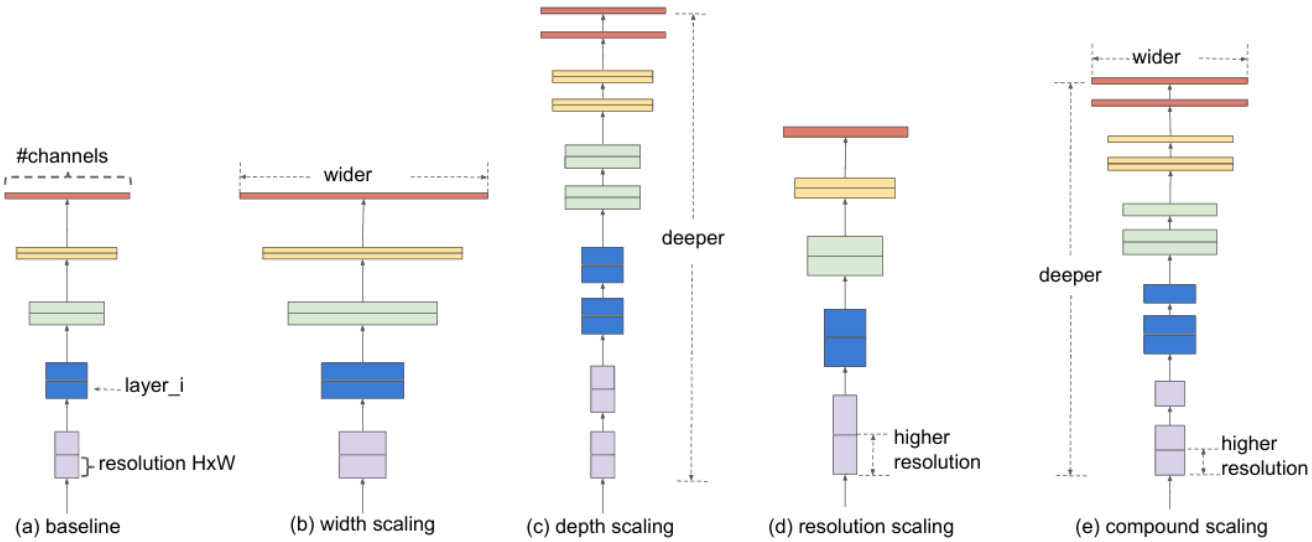
[۱] compound scaling

[۲] depth

[۳] width

[٤] resolution

$\gamma$ ،$\beta$ و $\alpha$ ثابت‌هایی هستند که با جستجوی شبکه‌ای روی یک مدل پایه کوچک به‌دست می‌آیند. $\phi$ ضریب مقیاس‌بندی است که اندازه کلی مدل را کنترل می‌کند و باید با توجه به منابع سخت افزاری انتخاب شود.




شکل ۱۰-۲. مقیاس بندی مدل - مدل a مدل پایه است. مدل های b، c و d مقیاس بندی هایی هستند که تنها یک بعد از عمق، عرض و وضوح شبکه را افزایش می دهند. مدل e معرف EfficientNet است که هر سه بعد عمق، عرض و وضوح را به طور یکسان افزایش می دهد.

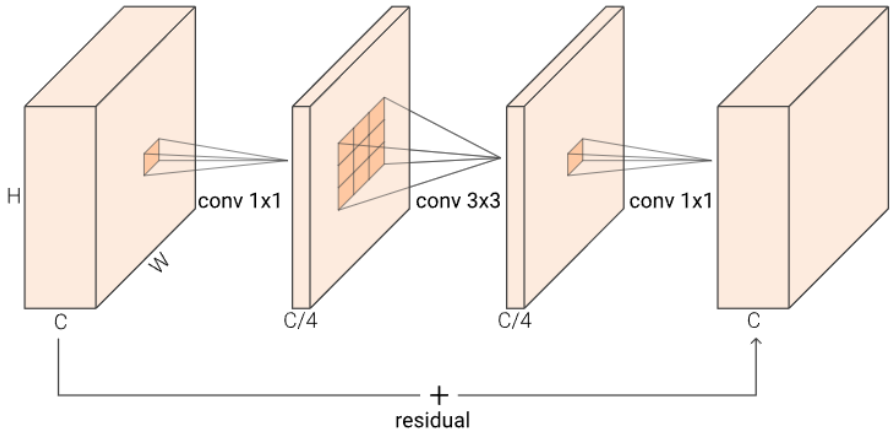
شکل ۱۱-۲، ساختار EfficientNet-B0 را نشان می دهد. ساختار جدیدی که در اینجا در مقایسه با CNN های عادی استفاده شده، بلوک MBConv می باشد.

| Stage $i$ | Operator $\hat{\mathcal{F}}_i$ | Resolution $\hat{H}_i \times \hat{W}_i$ | #Channels $\hat{C}_i$ | #Layers $\hat{L}_i$ |
|---|---|---|---|---|
| 1 | Conv3x3 | $224 \times 224$ | 32 | 1 |
| 2 | MBConv1, k3x3 | $112 \times 112$ | 16 | 1 |
| 3 | MBConv6, k3x3 | $112 \times 112$ | 24 | 2 |
| 4 | MBConv6, k5x5 | $56 \times 56$ | 40 | 2 |
| 5 | MBConv6, k3x3 | $28 \times 28$ | 80 | 3 |
| 6 | MBConv6, k5x5 | $14 \times 14$ | 112 | 3 |
| 7 | MBConv6, k5x5 | $14 \times 14$ | 192 | 4 |
| 8 | MBConv6, k3x3 | $7 \times 7$ | 320 | 1 |
| 9 | Conv1x1 & Pooling & FC | $7 \times 7$ | 1280 | 1 |

شکل ۱۱-۲. ساختار EfficientNet-B0

## ۲-۴-۲- بلوک MBConv

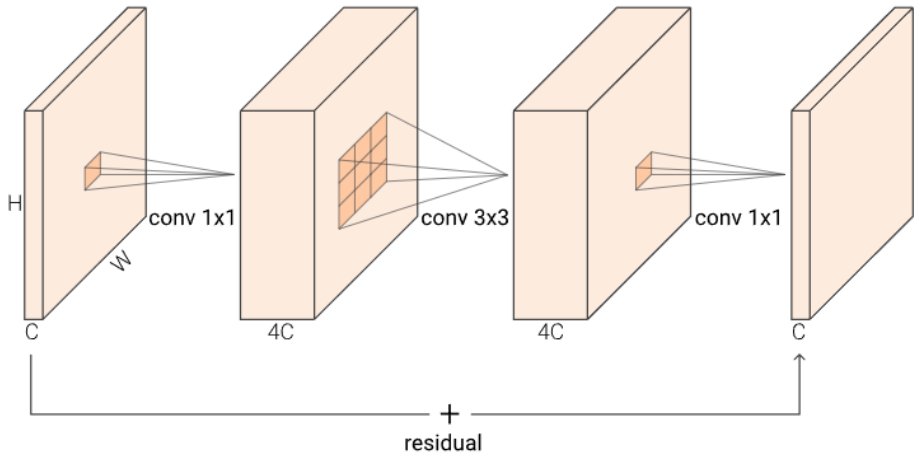
در سال ۲۰۱۷، گروهی از محققان گوگل مقاله‌ای منتشر کردند که یک معماری شبکه عصبی بهینه‌شده برای دستگاه‌های موبایل با پردازنده ضعیف تر معرفی می‌کند [6]. آن‌ها تلاش کردند مدلی بسازند که دقت بالا را حفظ کند و در عین حال پارامترها و عملیات‌های محاسباتی را به حداقل برساند. این ضروری بود تا شبکه‌های عمیق عصبی بتوانند بر روی گوشی‌های هوشمند اجرا شوند.
این معماری که «MobileNetV2» نامیده شده، بر پایه ایده استفاده از کانولوشن‌های جداپذیر عمق محور[۱] است که شامل یک کانولوشن عمق محور و سپس یک کانولوشن نقطه محور [۲]است.
بلوک پایه MBConv، بلوک باقیمانده است [7] . بلوک‌های باقیمانده [۳]اتصالاتی بین ابتدای و انتهای یک بلوک کانولوشنی ایجاد می‌کنند با یک اتصال میان‌بر [۴]با اضافه کردن فعال سازی [۵]اولیه و فعال سازی بعد از بلوک کانولوشنی، شبکه این فرصت را دارد که به فعال‌سازی‌های قبلی دسترسی داشته باشد که در بلوک کانولوشنی تغییر نکرده‌اند. این روش برای ساخت شبکه‌های عمیق ضروری است چرا که از پدیده ناپدید شدن گرادیان ها [۶]جلوگیری می کند. وقتی دقیق‌تر به اتصال میان‌بر نگاه می‌کنیم متوجه می‌شویم که بلوک باقیمانده معمولی ساختار پهن → باریک → پهن را در تعداد کانال‌ها دنبال می‌کند. ورودی دارای تعداد کانال زیاد است، سپس با یک کانولوشن ۱×۱ فشرده می‌شود تا تعداد پارامترهای کانولوشن ۳×۳ کاهش یابد. بعد از آن کانولوشن ۳×۳ اعمال می‌شود. نهایتاً برای جمع کردن وضعیت ابتدایی با خروجی، از یک کانولوشن ۱×۱ دیگر استفاده می‌کنند تا تعداد کانال‌ها دوباره افزایش یابد. شکل ۱۲-۲ بلوک باقیمانده را نشان می دهد.




شکل ۱۲-۲. ساختار بلوک باقیمانده

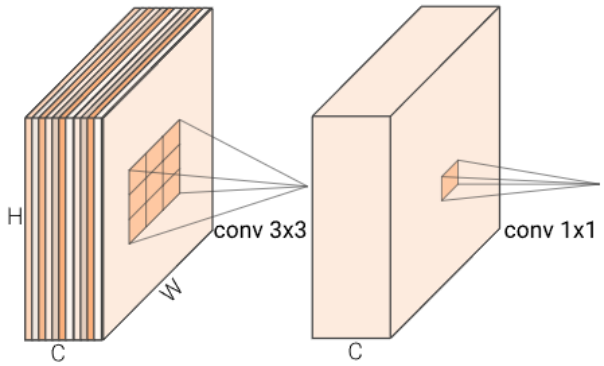
۱ depthwise separable convolutions
۲ pointwise
۳ residual
۴ skip connection
۵ activation
۶ vanishing gradient

در طرف دیگر، MobileNetV2 روشی معکوس دارد: ساختار باریک → پهن → باریک. ابتدا شبکه را با یک کانولوشن ۱×۱ گسترده‌تر می‌کنند چون کانولوشن عمق محور بعدی خودش تعداد زیادی از عملیات‌ها را کم می‌کند. سپس یک کانولوشن ۳×۳ تفکیکی عمقی اعمال می‌شود. بعد از آن یک کانولوشن ۱×۱ دیگر شبکه را فشرده می‌کند تا تعداد کانال‌ها با تعداد اولیه ورودی هم‌خوانی داشته باشد. شکل ۱۳-۲ بلوک باقیمانده معکوس با کانولوشن عمقی را یا همان MBconv را نشان می دهد.




شکل ۱۳-۲. ساختار بلوک باقیمانده معکوس شده با کانولوشن جداپذیر عمق محور یا همان MBConv

کانولوشن جداپذیر عمق‌محور از یک ترفند استفاده می‌کنند تا یک کانولوشن معمولی ۳×۳ را به دو کانولوشن تقسیم کنند و بدین ترتیب تعداد پارامترها را کاهش دهند. کانولوشن اول یک فیلتر ۳×۳ را به هر کانال ورودی اعمال می‌کند، و کانولوشن دوم یک فیلتر ۱×۱ را روی تمام کانال‌ها اعمال می‌کند. اگر محاسبه را انجام دهیم، می بینیم که این همان کاری است که یک کانولوشن معمولی ۳×۳ انجام می‌دهد، ولی با صرفه‌جویی در تعداد پارامترها. شکل ۱۴-۲ کانولوشن جداپذیر عمق‌محور را نشان می دهد.




شکل ۱۴-۲. ساختار بلوک باقیمانده معکوس شده با کانولوشن جداپذیر عمق محور یا همان MBConv

دلیل استفاده از توابع فعال‌سازی غیرخطی در شبکه‌های عصبی این است که ضرب ماتریس‌ها بدون این تابع فعال‌سازی نمی‌توانند به یک عملیات عددی ساده کاهش یابند. این امکان را به شبکه می‌دهد که لایه‌های زیادی داشته باشد. در عین حال، تابع ReLU که معمولاً استفاده می‌شود مقادیر کمتر از صفر را دور می‌اندازد. این از دست رفتن اطلاعات را می‌توان با افزایش تعداد کانال‌ها جبران کرد تا ظرفیت شبکه بیشتر شود. با بلوک‌های باقیمانده معکوس، ما لایه‌هایی را که اتصالات میان‌بر در آن‌ها برقرارند فشرده می‌کنیم. این به عملکرد شبکه لطمه می‌زند. نویسندگان مفهوم تنگنای خطی [1] را معرفی کردند که آخرین کانولوشن در بلوک باقی‌مانده فعال‌سازی غیرخطی ندارد، یعنی خروجی خطی دارد پیش از آنکه به فعال‌سازی اولیه افزوده شود.

## ۵-۲- فرآیند آموزش

ما از یک فرایند آموزش چندمرحله‌ای استفاده می‌کنیم که در هر مرحله تنظیمات و مجموعه‌داده های متفاوتی به کار می‌رود.

### ۱-۵-۲- مرحله پیش آموزش

ما دریافتیم که طرح‌های برچسب‌گذاری بین مجموعه‌داده‌ها ناسازگار هستند، بنابراین تصمیم گرفتیم از بزرگ‌ترین مجموعه‌داده (داده‌های سال ۲۰۱۵) برای پیش‌آموزش شبکه‌های عصبی خود استفاده کنیم. استفاده از یادگیری انتقالی ممکن است، زیرا ویژگی‌های طبیعی رتینوپاتی دیابتی بین افراد مختلف یکسان است و به مجموعه‌داده وابسته نیست. علاوه بر این، مجموعه‌داده‌های مختلف با تجهیزات متفاوت جمع‌آوری شده‌اند. گنجاندن این آگاهی در مدل، توانایی تعمیم آن را افزایش می‌دهد و با کاهش حساسیت به نویز ابزار، اهمیت ویژگی‌های طبیعی را بالاتر می‌برد.

ما استخراج‌کننده ویژگی را با وزن‌های شبکه عصبی کانولوشنی که روی Imagenet پیش‌آموزش دیده است، مقداردهی اولیه می‌کنیم. سرها با وزن‌های تصادفی مقداردهی اولیه می‌شوند. ما مدل را به مدت ۱۰ اپوک روی داده های اپتوس ۲۰۱۵ با استفاده از SGD با مینی‌بچ و برنامه تغییر نرخ یادگیری به صورت Cosine-Annealing آموزش می‌دهیم [8] .

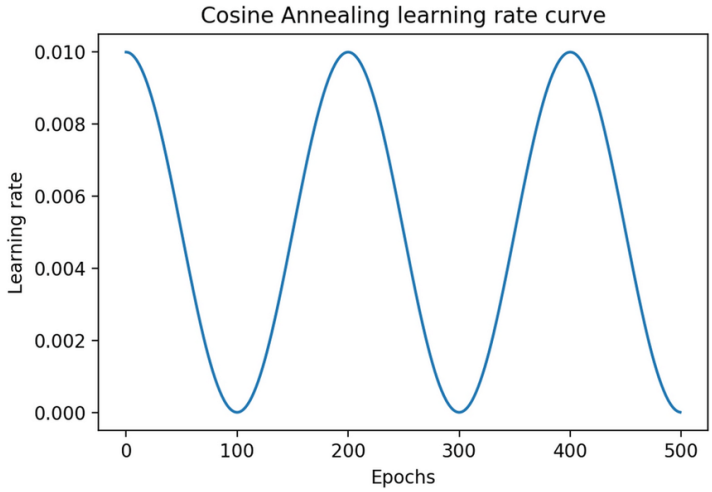
[1] linear bottleneck

Cosine Annealing یک روش تغییر نرخ یادگیری در طول آموزش مدل‌های یادگیری عمیق است. در این روش، نرخ یادگیری به تدریج از یک مقدار اولیه بزرگ $(\eta_{max})$ کاهش یافته و به مقدار نهایی کوچک $(\eta_{min})$ می‌رسد، به طوری که کاهش آن بر اساس تابع کسینوس انجام می‌شود.

نرخ یادگیری در گام t از فرمول زیر محاسبه می‌شود:

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\frac{t\pi}{T})$$

که در آن T تعداد کل گام‌های آموزشی یا اپوک‌ها است. این روش باعث می‌شود آموزش پایدارتر باشد و امکان همگرایی دقیق‌تر به مینیمم تابع خطا فراهم شود. نسخه‌های چرخه‌ای این روش [1] با افزایش مجدد نرخ یادگیری در دوره‌های بعدی، امکان فرار از مینیمم‌های محلی را نیز فراهم می‌کنند. در این مرحله از نسخه چرخشی استفاده شده است. شکل ۱۵-۲ مقدار این مدل نرخ یادگیری را نشان می دهد.




شکل ۱۵-۲. نرخ یادگیری Cosine Annealing

[1] Cosine Annealing with Restarts

از الگوریتم SGD [1] که در این مرحله استفاده شده است که از شبه کد زیر استفاده کرده است [9] :

**input** : $\gamma$ (lr), $\theta_0$ (params), $f(\theta)$ (objective), $\lambda$ (weight decay),
$\mu$ (momentum), $\tau$ (dampening), *nesterov*, *maximize*

**for** $t = 1$ **to** ... **do**
  **if** *maximize* :
    $g_t \leftarrow -\nabla_\theta f_t(\theta_{t-1})$
  **else**
    $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$
  **if** $\lambda \neq 0$
    $g_t \leftarrow g_t + \lambda\theta_{t-1}$
  **if** $\mu \neq 0$
    **if** $t > 1$
      $\mathbf{b}_t \leftarrow \mu\mathbf{b}_{t-1} + (1-\tau)g_t$
    **else**
      $\mathbf{b}_t \leftarrow g_t$
    **if** *nesterov*
      $g_t \leftarrow g_t + \mu\mathbf{b}_t$
    **else**
      $g_t \leftarrow \mathbf{b}_t$
  $\theta_t \leftarrow \theta_{t-1} - \gamma g_t$

**return** $\theta_t$

هدف الگوریتم بروزرسانی پارامتر های وزن یا $\theta_t$ در کل شبکه عصبی می باشد. این پارامتر ها، پارامتر های قابل آموزش نیز نامیده می شوند. تعداد کل این پارامتر ها در این شبکه حدود ۶،۸۵۰،۲۷۳ بود که در بخشی دیگر به تشریح این مورد خواهیم پرداخت.

به طوری کلی چند خط اولیه این شبه کد برای پیاده سازی الگوریتم کافی هستند. قسمت های بعد برای رسیدن همگرایی سریع تر و پایداتر می باشند. در ساده ترین فرم اگر $f$ تابع مقدار خطا و $\gamma$ مقدار نرخ یادگیری باشد، رابطه زیر را برای به دست آوردن هر پارامتر قابل آموزش در مرحله $t$ داریم:

$$\theta_t = \theta_{t-1} - \gamma\nabla_\theta f(\theta_{t-1})$$

برای جلوگیری از بیش برازش، می توان در همین الگوریتم از پارامتر $\lambda$ یا کاهش وزن [2] استفاده کرد که افزایش آن از بیش برازش جلوگیری خواهد کرد، اما افزایش بیش از حد آن نیز به یادگیری مدل آسیب خواهد زد.

[1] stochastic gradient descent

پارامتر دیگر $\mu$ یا مومنتوم [۱]می باشد. مومنتوم یک میانگین متحرک از گرادیان‌های گذشته را نگه می‌دارد. این کار باعث صاف‌تر شدن به‌روزرسانی‌ها شده و می‌تواند سرعت همگرایی را افزایش دهد.
دمپینگ[2] *(τ)* اثر گرادیان جدید را در محاسبه مومنتوم کاهش می‌دهد.
پارامتر دیگر نستروف[۳] است. مومنتوم نستروف با اضافه کردن یک جمله اضافی (در مقایسه با مومنتوم معمول)، به جلو نگاه می‌کند که به اصلاح بیش‌پیشروی[۴] کمک می‌کند .در پیاده سازی این الگوریتم از همه این پارامترها استفاده شده است.

## انتخاب تابع مقدار خطا

در این الگوریتم تابع مقدار خطا برای هر سر، یک نوع خاص انتخاب شده است. هر سر در حال کمینه کردن تابع هزینه خودش است: کراس انتروپی [۵]برای سر دسته‌بندی، کراس انتروپی باینری برای سر رگرسیون ترتیبی، و میانگین خطای مطلق [۶]برای سر رگرسیون.

تابع خطای کراس انتروپی باینری به صورت زیر می باشد:

$$BCE\ loss = -\frac{1}{N}\sum_{i=1}^{N}(y_i * \log(p_i) + (1-y_i) * \log(1-p_i))$$

که در اینجا N تعداد داده ها، $y_i$ برچسب واقعی داده شماره $i$ ام (۰ یا ۱) و $p_i$ برچسب پیشبینی شده داده شماره $i$ ام توسط مدل است.
تا بع خطای کراس انتروپی به صورت زیر می باشد:

$$CE\ loss = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{C}(y_{i,j} * \log(p_{i,j}))$$

که در اینجا N تعداد داده ها و C تعداد کلاس ها است. مقدار $y_{i,j}$ ۱ است اگر کلاس $j$ برای داده شماره $i$ درست باشد، در غیر این صورت ۰ است. مقدار $p_{i,j}$ احتمال متعلق بودن داده شماره $i$ به کلاس شماره $j$ می باشد که توسط مدل پیشبینی می شود.

---

[۲] weight decay
[۱] momentum
[2] damping
[۳] Nesterov
[۴] overshooting
[۵] Cross Entropy
[۶] mean absolute error

تابع میانگین خطای مطلق به صورت زیر می باشد:

$$MSE\ loss = \frac{1}{N}\sum_{i=1}^{N}|y_i - p_i|$$

که در اینجا N تعداد داده ها، $y_i$ برچسب واقعی داده شماره $i$ ام (۰ یا ۱) و $p_i$ برچسب پیشبینی شده داده شماره $i$ ام توسط مدل است.

بعد از پیش آموزش، ما از وزن‌های کدکننده را به عنوان مقداردهی اولیه برای مراحل بعدی استفاده می‌کنیم.

در آزمایش‌های انجام شده توسط نویسندگان مقاله، مشاهده شد که وقتی وزن‌های سر ها را قبل از آموزش اصلی [۱]با مقداردهی اولیه تصادفی جایگزین می‌کنیم، معیارها به طور مداوم بهبود می‌یابند؛ بنابراین، وزن‌های آموزش‌دیده سر ها را کنار گذاشته شدند. ما نیز همین دستور را در پیش گرفتیم.

## ۲-۵-۲- مرحله آموزش اصلی

آموزش اصلی روی داده‌های APTOS ۲۰۱۹، IDRiD و MESSIDOR (به‌صورت ترکیبی) انجام شد. البته فقط از ۸۰ درصد داده های APTOS ۲۰۱۹ استفاده شد و ۲۰ درصد دیگر به عنوان مجموعه داده آزمایش کنار گذاشته شده بود. ما با شروع از وزن‌هایی که در مرحله پیش‌آموزش به دست آمده بودند، آموزش اصلی را شروع کردیم.

در مقاله اقتباسی از اعتبارسنجی متقاطع k بخشی [۲]را اجرا شد، که k=5 انتخاب شده است. در این روش k مدل باید آموزش داده شوند و برای هر مدل مجموعه داده ها به k بخش تقسیم می شوند. k-1 عدد از این k بخش به طور تصادفی انتخاب می شود و این مجموعه داده برای آموزش مدل مورد استفاده قرار می گیرد. بخش k ام که باقی مانده به عنوان مجموعه داده اعتبارسنجی [۳]مورد استفاده قرار می گیرد. برای هر مدل این کار را انجام می دهیم و معیار های ارزیابی را میانگین این k مدل در نظر می گیریم. این روش به ما معیار های ارزیابی واقع گرایانه تری می دهد ولی ما از این روش به دلیل کمبود منابع سخت افزاری و افزایش طول مدت آموزش استفاده نکردیم.

در نهایت معیار ارزیابی روی مجموعه کنار گذاشته شده APTOS ۲۰۱۹ به عنوان گزارش نهایی انتخاب می شود.

در این مرحله، ما توابع خطا را برای سرهای مدل تغییر دادیم. در مقاله اقتباسی برای سر دسته بندی از خطای کانونی[۴]، برای سر رگرسیون ترتیبی از خطای کانونی باینری و برای سر رگرسیون از

---

[۱] main training

[۲] k-fold cross validation

[۳] validation dataset

[۴] Focal Loss

خطای میانگین مربعات [1]استفاده شد. در این پروژه ما از سر دسته بندی استفاده کردیم بنابراین فقط از تابع خطای کانونی استفاده شد.

## تابع خطای کانونی

تابع خطای خطای کانونی یک راه حل پیشنهادی برای حل مشکل توزیع نامتوازن بین کلاس ها می باشد [11]. تعریف این تابع با تابع کراس انتروپی باینری شروع می شود که تابع زیر می باشد.

$$BCE(p,y) = \begin{cases} -\log(p) & if\ y = 1 \\ -\log(1-p) & otherwise \end{cases}$$

که در اینجا $y$ کلاس واقعی است که ۰ یا ۱ است و $p$ احتمالی است که مدل برای داشتن برچسب $y = 1$ پیش بینی می کند. حال برای ساده سازی این تابع را به صورت زیر می نویسیم:

$$p_t = \begin{cases} p & if\ y = 1 \\ 1-p & otherwise \end{cases}$$

و می نویسیم: $CE(p,y) = CE(p_t) = -\log(p_t)$

در مجموعه داده هایی که توزیع نامتوازن شدیدی وجود دارد، تابع خطای کراس انتروپی تحت‌تأثیر کلاس با فراوانی بیشتر قرار میگیرد. نمونه‌های فراوان تر که به‌راحتی طبقه‌بندی می‌شوند، بخش عمده خطا را تشکیل داده و بر گرادیان غالب می‌شوند. برای حل این مشکل تابع خطای کانونی به صورت زیر ارائه می شود:

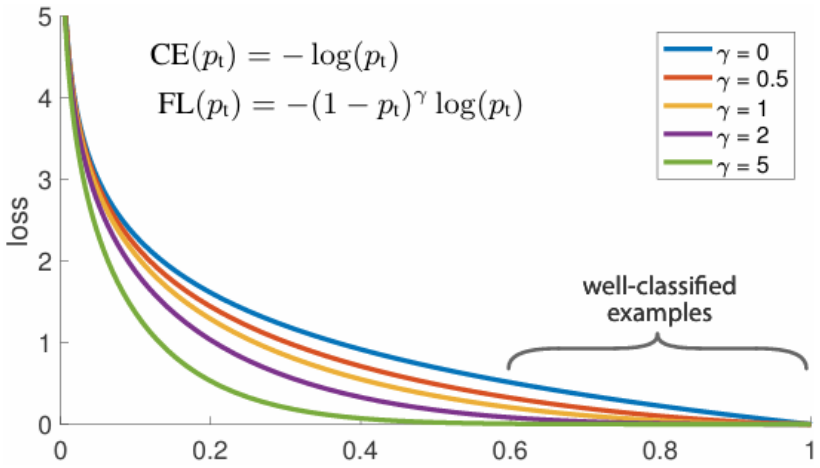
$$FL(p_t) = -\alpha_t(1-p_t)^{\gamma}\log(p_t)$$

که در اینجا $\alpha_t$ پارمتر وزن است که برای کلاس $y = 1$ وزن $\alpha$ را داریم و برای کلاس $y = 0$ وزن $1-\alpha$ را داریم. برای حل مشکل نامتوازن بودن کلاس ها، می توان این پارامتر را برابر معکوس فراوانی کلاس قرار داد. سپس پارامتر $\gamma$ را داریم. این پارامتر تمرکز [2]است که معمولا برابر مقداری از بازه ۰ تا ۵ انتخاب می شود. این پارامتر به مدل کمک می کند تا نمونه های سخت تر از نظر کلاس بندری را بهتر تشخیص دهد. نمونه هایی که مدل به آسانی کلاس بندی می کند مقدار $p$ بالاتری دارند و وقتی خطاهای این نمونه ها با هم جمع می شوند، این مجموع می تواند مقدار قابل توجهی باشد و خطای نمونه های کمیاب تر را کم رنگ تر جلوه دهد. پارامتر $\gamma$ خطای موارد آسان تر را کاهش می دهد تا مدل روی نمونه های سخت تر تمرکز بیشتری داشته باشد. تابع $FL(p_t)$ را به همین ترتیب برای مسئله کلاس بندی چند کلاسه نیز می توان تعمیم داد.

[1] mean-squared error

[2] foucs

شکل تابع خطای focal loss را با binary cross entropy مقایسه می کند.




شکل ۱۶-۲. مقایسه تابع خطای کانونی و کراس انتروپی باینری
نمونه هایی که مدل آن ها را احتمال بالاتری برچسب می زند با تابع خطای کانونی مقدار خطای کمتری دارند.

همان طور که گفتیم برای سر رگرسیون از تابع خطای میانگین مربعات استفاده می شود:

$$MSE\ loss = \frac{1}{N}\sum_{i=1}^{N}(y_i - p_i)^2$$

که در اینجا N تعداد داده ها، $y_i$ برچسب واقعی داده شماره $i$ ام (۰ یا ۱) و $p_i$ برچسب پیشبینی شده داده شماره $i$ ام توسط مدل است.

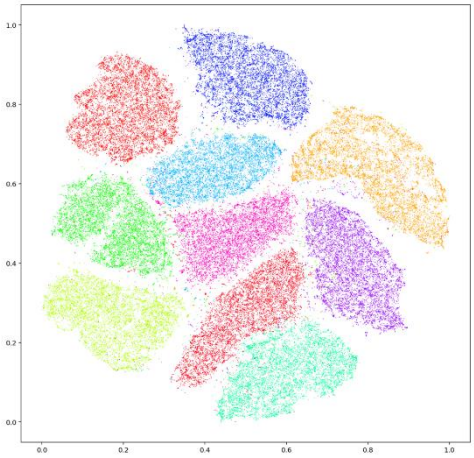
در مقاله اقتباسی هر بخش (fold) به مدت ۷۵ دوره (epoch) با استفاده از بهینه ساز آدام اصلاح شده[۱] و برنامه زمان‌بندی نرخ یادگیری کاهشی کسینوسی آموزش داده شد. ما به جای ۷۵ دوره، ۴۰ دوره برای مدل ۵ دسته ای و ۳۰ دوره برای مدل باینری برای سرعت آموزش بیشتر انتخاب کردیم. نخست در حالی که سرهای جدید در حالت تصادفی هستند، آموزش کدکننده را به مدت پنج دوره غیرفعال کردیم تنها سرهای مدل را آموزش دادیم. علت این کار این است که مدل ممکن است وزن های ذخیره شده در مرحله پیش آموزش را فراموش کند و عملا آموزش از اول آغاز شود. به این پدیده فراموش کردن وزن ها فراموشی فاجعه بار[۲] می گویند.

[۱] Rectified Adam

[۲] catastrophic forgetting

در طول آموزش اصلی، ما قابلیت جداسازی در فضای ویژگی تولیدشده توسط کدکننده را بررسی کردیم. ما بردارهای توکار دوبعدی [1] را با استفاده از روش t-SNE تولید کرده و آن‌ها را در مرحله اعتبارسنجی برای کنترل دستی عملکرد آموزش، نمایش دادیم.

t-SNE [2] یک روش کاهش بُعد غیرخطی است که برای نمایش داده‌های با بُعد بالا در فضای ۲ بعدی یا ۳ بعدی استفاده می‌شود تا بتوانیم ساختارها، خوشه‌ها و روابط پنهان در داده را ببینیم [10] . ایده اصلی این الگوریتم این است که نقاطی که در فضای اصلی (بُعد بالا) به هم نزدیک هستند، در فضای کاهش‌یافته هم نزدیک باقی بمانند و برعکس، نقاطی که فاصله زیادی دارند، در فضای کاهش‌یافته نیز فاصله‌دار باشند. برای این کار، t-SNE ابتدا فاصله بین هر جفت نقطه را به یک مقدار احتمال شباهت تبدیل می‌کند؛ یعنی اگر دو نقطه به هم نزدیک باشند، احتمال همسایگی آن‌ها زیاد است و اگر دور باشند، این احتمال کم است. سپس همین کار را برای فضای کاهش‌یافته انجام می‌دهد و سعی می‌کند چینش نقاط در این فضای جدید طوری باشد که توزیع احتمالات شباهت بین نقاط در این فضا تا جای ممکن شبیه به توزیع فضای اصلی باشد. فضای اصلی (بُعد بالا) در هر مرحله یک واحد کاهش بُعد دارد که این کار با الگوریتم گرادیان کاهشی انجام می شود. به همین علت روش t-SNE از نظر محسباتی پرهزینه است. روش های دیگری مانند PCA [3] نیز برای مسئله کاهش بُعد داده ها وجود دارند که از نظر محاسباتی کم هزینه ترند ولی برای داده های غیرخطی به عملکرد روش t-SNE نمی رسند. نکته مهم دیگری که درباره t-SNE وجود دارد این است که ابعاد نهایی که معمولا ۲ بعدی یا ۳ بعدی هستند، هیچ معنایی ندارند و دیگر ویژگی های اولیه در فضای اصلی از بین می روند. منظور از ویژگی های اولیه ویژگی هایی است که معنا دارند، برای مثال در مسئله پیشبینی قیمت خانه، قدمت ساخت خانه یک ویژگی اولیه و بامعنا است.



شکل ۱۲ – کاهش ابعاد با روش t-SNE برای مجموعه داده MNIST که شامل عکس های ارقام ۰ تا ۹ می باشد به طوری هر رنگ مربوط به یک رقم مشخص است. داده های مربوط به هر رقم کنار هم به صورت خوشه قرار گرفته اند.

[1] 2D embeddings

[2] t-distributed Stochastic Neighbor Embedding

[3] Principal Component Analysis

## ۳-۵-۲- مرحله پسا آموزش

مرحله پسا آموزش آخرین مرحله آموزش مدل طبق روش مقاله اقتباسی است. در این پروژه به دلیل انتخاب معماری ساده تر مدل و حذف سر رگرسیون نهایی، نیازی به انجام این مرحله نبود. در ادامه این مرحله را شرح می دهیم.

در مرحله پسا آموزش، تنها مدل رگرسیون خطی را بر خروجی‌های سرهای مختلف برازش داده می شود. آزمایش ها نشان داد که ضروری است این مدل در مراحل قبلی به‌روزرسانی نشود، زیرا در غیر این صورت، به یک مینیمم محلی غیر بهینه همگرا می‌شود که در آن وزن‌های دو سر نزدیک به صفر هستند. این ضرایب از به‌روزرسانی گرادیان‌های وزن‌های سرهای متناظر جلوگیری کرده و بیشتر مانع همگرایی شبکه می‌شوند. وزن‌های اولیه برای هر سر برابر با $\frac{1}{3}$ تنظیم شد و سپس به مدت پنج دوره برای کمینه‌کردن تابع خطای میانگین مربعات آموزش داده شد.

## ۴-۵-۲- منظم سازی و مرحله تجمیع

در زمان آموزش، ما مدل‌های خود را برای افزایش پایداری [۱]منظم‌سازی [۲]می‌کنیم. از روش‌های متداولی مانند کاهش وزن استفاده می‌کنیم. همچنین، با استفاده از هموارسازی برچسب‌ها [۳]شبکه را به دلیل پیش‌بینی‌های بیش‌ازحد مطمئن جریمه می‌کنیم.

علاوه بر هموارسازی برچسب‌ها برای سرهای دسته‌بندی و رگرسیون ترتیبی، ما یک طرح هموارسازی برچسب برای سر رگرسیون خطی پیشنهاد می‌کنیم. این روش می‌تواند زمانی استفاده شود که معلوم باشد مقادیر هدف، گسسته هستند. در این روش، ما به مقادیر هدف گسسته، نویز تصادفی یکنواخت اضافه می‌کنیم:

$$T_s = T + \Delta$$
$$\Delta = \mathcal{U}(a, b)$$

در اینجا $T_s$ برچسب هدف هموارشده، $T$ برچسب اصلی، و $\mathcal{U}$ توزیع یکنواخت است. در این حالت داریم:

$$-a = b = \frac{T_i - T_{i+1}}{3}$$

که در اینجا $T_i$ و $T_{i+1}$ برچسب‌های هدف گسسته و همسایه هستند.

---

[۱] robustness

[۲] regularize

[۳] label smoothing

با اعمال این روش هموارسازی، می‌توانیم اهمیت برچسب‌گذاری نادرست را کاهش دهیم. البته ما در این پروژه فقط از سر دسته بندی استفاده کردیم و طرح هموارسازی برای این سر استفاده شد. برای مثال اگر یکی از عکس ها برچسب واقعی دسته شماره ۲ را داشته باشد، بردار آن به صورت $[0, 0, 1, 0, 0]$ خواهد بود. حال اگر طرح هموارسازی را اجرا کنیم، این بردار به $[\varepsilon, \varepsilon, 1 - k\varepsilon, \varepsilon, \varepsilon]$ تغییر خواهد یافت که در اینجا $\varepsilon$ عددی کوچک (معمولا بین ۰ تا ۰.۳) و $k$ برابر با تعداد دسته ها (۵) می باشد.

روش تجمیع [۱]باعث بهبود مدل نهایی و در نتیجه بهبود معیار های ارزیابی می شود، اما از نظر حجم محاسبات پرهزینه است و ما در این پروژه از آن صرف نظر کردیم. در ادامه روش مقاله اقتباسی برای تجمیع مدل ها را توضیح می دهیم.

طبق مقاله اقتباسی، برای امتیازدهی نهایی مدل‌هایی با ۳ معماری کدکننده در وضوح‌های مختلف که بهترین عملکرد را روی مجموعه داده نگهداری‌شده[2] داشتند، تجمیع شدند. این ۳ مدل عبارتند از: EfficientNet-B4 با وضوح ۳۸۰×۳۸۰، EfficientNet-B5 با وضوح ۴۵۶×۴۵۶ و SE-ResNeXt50 با وضوح‌های ۳۸۰×۳۸۰ و ۵۱۲×۵۱۲.

بهترین راهکار یک تجمیع ۲۰ مدل شامل ۴ معماری × ۵ (تا اعتبارسنجی متقاطع ۵ بخشی) همراه با افزونه‌های زمان آزمون [۳]شامل: وارون‌سازی افقی، وارون‌سازی عمودی، جابه‌جایی ماتریسی [۴]، چرخش، و بزرگ‌نمایی [۵]بود. در مجموع، این روش برای هر تصویر فوندوس، ۲۰۰ پیش‌بینی تولید کرد. این پیش‌بینی‌ها با استفاده از میانگین برش‌خورده [۶]با مقدار ۰.۲۵ ترکیب شدند تا داده‌های پرت [۷]حاصل از مدل‌های احتمالا بیش‌برازش‌شده حذف شوند. میانگین برش‌خورده روشی است برای فیلتر کردن داده‌های پرت به منظور کاهش واریانس. در این روش، ابتدا مقادیر پیش‌بینی مرتب می‌شوند. سپس ٪۲۵ از مقادیر کوچک‌تر و ٪۲۵ از مقادیر بزرگ‌تر حذف می‌گردند و میانگین از ٪۵۰ داده‌های میانی محاسبه می‌شود.

---

۱ ensemble

2 holdout dataset

۳ test-time augmentations (TTA)

۴ transpose

۵ zoom

۶ 0.25-trimmed mean

۷ outliers

## ۶-۲- نتیجه گیری

در این فصل، مجموعه‌داده‌ها و معیارهای ارزیابی مورد استفاده معرفی شدند و مراحل پیش‌پردازش داده‌ها تشریح گردیدند. سپس معماری شبکه و کدکننده مبتنی بر یادگیری انتقالی توضیح داده شد و در ادامه، فرایند آموزش مدل در دو مرحله پیش‌آموزش و آموزش اصلی همراه با توابع خطا بررسی شد. در پایان نیز به تکنیک‌های منظم‌سازی و تجمیع اشاره گردید. مباحث این فصل چارچوب کلی طراحی و آموزش مدل پیشنهادی را مشخص کرده و زمینه را برای ارائه نتایج و ارزیابی عملکرد مدل در فصل بعد فراهم می‌سازد.

# فصل سوم: نتایج شبیه‌سازی یا پیاده‌سازی

## ۳-۱- مقدمه

در این فصل، فرایند آموزش مدل شرح داده می‌شود. ابتدا با اجرای کد، گزارشی از معماری مدل استخراج شده و ساختار آن بررسی می‌گردد. سپس فرایند پیش‌آموزش آغاز شده و پارامترهای کلیدی همراه با عملیات مهم انجام‌شده توضیح داده می‌شوند. پس از اتمام پیش‌آموزش، نمودارهای مربوط به مقدار خطا و دقت برای دو مدل دودسته‌ای و پنج‌دسته‌ای رسم می‌شوند. در ادامه، وزن‌های آموزش یافته ذخیره شده و آموزش اصلی مدل آغاز می‌گردد. در این مرحله نیز پارامترهای مهم انتخاب و عملیات کلیدی انجام‌شده تشریح می‌شوند. پس از پایان آموزش اصلی، نمودارهای خطا و دقت برای هر دو مدل رسم شده و در نهایت، مدل نهایی روی مجموعه داده آزمایشی ارزیابی می‌شود و نتایج حاصل با نتایج مقاله اقتباسی مقایسه می‌گردد.

## ۳-۲- فرآیند آموزش

در این مرحله دو مدل را آموزش می دهیم. مدل اول برای دسته بندی باینری و مدل دوم برای دسته بندی ۵ دسته ای می باشد. معماری این دو مدل کاملا یکسان هستند، فقط سر نهایی آن ها تفاوت دارد. تفاوت دیگر این دو مدل در مرحله آموزش اصلی است که برای مدل باینری تابع خطای BCE را انتخاب کردیم ولی برای مدل ۵ دسته از Focal Loss استفاده کردیم.

قبل از شروع به آموزش، خلاصه از معماری شبکه تهیه کردیم:

| Layer (type (var_name)) | Input Shape | Output Shape | Param # | Trainable |
|---|---|---|---|---|
| OneHeadModel (OneHeadModel) | [32, 3, 240, 240] | [32] | -- | True |
| ├─Sequential (encoder) | [32, 3, 240, 240] | [32, 1280, 8, 8] | -- | True |
| │ └─Conv2dNormActivation (0) | [32, 3, 240, 240] | [32, 32, 120, 120] | -- | True |
| │ │ └─Conv2d (0) | [32, 3, 240, 240] | [32, 32, 120, 120] | 864 | True |
| │ │ └─BatchNorm2d (1) | [32, 32, 120, 120] | [32, 32, 120, 120] | 64 | True |
| │ │ └─SiLU (2) | [32, 32, 120, 120] | [32, 32, 120, 120] | -- | -- |
| │ └─Sequential (1) | [32, 32, 120, 120] | [32, 16, 120, 120] | -- | True |
| │ │ └─MBConv (0) | [32, 32, 120, 120] | [32, 16, 120, 120] | 1,448 | True |
| │ │ └─MBConv (1) | [32, 16, 120, 120] | [32, 16, 120, 120] | 612 | True |
| │ └─Sequential (2) | [32, 16, 120, 120] | [32, 24, 60, 60] | -- | True |
| │ │ └─MBConv (0) | [32, 16, 120, 120] | [32, 24, 60, 60] | 6,004 | True |
| │ │ └─MBConv (1) | [32, 24, 60, 60] | [32, 24, 60, 60] | 10,710 | True |
| │ │ └─MBConv (2) | [32, 24, 60, 60] | [32, 24, 60, 60] | 10,710 | True |
| │ └─Sequential (3) | [32, 24, 60, 60] | [32, 40, 30, 30] | -- | True |
| │ │ └─MBConv (0) | [32, 24, 60, 60] | [32, 40, 30, 30] | 15,350 | True |
| │ │ └─MBConv (1) | [32, 40, 30, 30] | [32, 40, 30, 30] | 31,290 | True |
| │ │ └─MBConv (2) | [32, 40, 30, 30] | [32, 40, 30, 30] | 31,290 | True |
| │ └─Sequential (4) | [32, 40, 30, 30] | [32, 80, 15, 15] | -- | True |
| │ │ └─MBConv (0) | [32, 40, 30, 30] | [32, 80, 15, 15] | 37,130 | True |
| │ │ └─MBConv (1) | [32, 80, 15, 15] | [32, 80, 15, 15] | 102,900 | True |
| │ │ └─MBConv (2) | [32, 80, 15, 15] | [32, 80, 15, 15] | 102,900 | True |
| │ │ └─MBConv (3) | [32, 80, 15, 15] | [32, 80, 15, 15] | 102,900 | True |
| │ └─Sequential (5) | [32, 80, 15, 15] | [32, 112, 15, 15] | -- | True |
| │ │ └─MBConv (0) | [32, 80, 15, 15] | [32, 112, 15, 15] | 126,004 | True |
| │ │ └─MBConv (1) | [32, 112, 15, 15] | [32, 112, 15, 15] | 208,572 | True |
| │ │ └─MBConv (2) | [32, 112, 15, 15] | [32, 112, 15, 15] | 208,572 | True |
| │ │ └─MBConv (3) | [32, 112, 15, 15] | [32, 112, 15, 15] | 208,572 | True |
| │ └─Sequential (6) | [32, 112, 15, 15] | [32, 192, 8, 8] | -- | True |
| │ │ └─MBConv (0) | [32, 112, 15, 15] | [32, 192, 8, 8] | 262,492 | True |
| │ │ └─MBConv (1) | [32, 192, 8, 8] | [32, 192, 8, 8] | 587,952 | True |
| │ │ └─MBConv (2) | [32, 192, 8, 8] | [32, 192, 8, 8] | 587,952 | True |
| │ │ └─MBConv (3) | [32, 192, 8, 8] | [32, 192, 8, 8] | 587,952 | True |
| │ │ └─MBConv (4) | [32, 192, 8, 8] | [32, 192, 8, 8] | 587,952 | True |
| │ └─Sequential (7) | [32, 192, 8, 8] | [32, 320, 8, 8] | -- | True |
| │ │ └─MBConv (0) | [32, 192, 8, 8] | [32, 320, 8, 8] | 717,232 | True |
| │ │ └─MBConv (1) | [32, 320, 8, 8] | [32, 320, 8, 8] | 1,563,600 | True |
| │ └─Conv2dNormActivation (8) | [32, 320, 8, 8] | [32, 1280, 8, 8] | -- | True |
| │ │ └─Conv2d (0) | [32, 320, 8, 8] | [32, 1280, 8, 8] | 409,600 | True |
| │ │ └─BatchNorm2d (1) | [32, 1280, 8, 8] | [32, 1280, 8, 8] | 2,560 | True |
| │ │ └─SiLU (2) | [32, 1280, 8, 8] | [32, 1280, 8, 8] | -- | -- |
| ├─AdaptiveMaxPool2d (global_max_pool) | [32, 1280, 8, 8] | [32, 1280, 1, 1] | -- | -- |
| ├─AdaptiveAvgPool2d (global_avg_pool) | [32, 1280, 8, 8] | [32, 1280, 1, 1] | -- | -- |
| ├─BatchNorm1d (batch_norm_1) | [32, 1280] | [32, 1280] | 2,560 | True |
| ├─BatchNorm1d (batch_norm_2) | [32, 1280] | [32, 1280] | 2,560 | True |
| ├─Sequential (dense1) | [32, 2560] | [32, 128] | -- | True |
| │ └─Dropout (0) | [32, 2560] | [32, 2560] | -- | -- |
| │ └─Linear (1) | [32, 2560] | [32, 128] | 327,808 | True |
| │ └─ReLU (2) | [32, 128] | [32, 128] | -- | -- |
| │ └─Dropout (3) | [32, 128] | [32, 128] | -- | -- |
| ├─Sequential (classification_head) | [32, 128] | [32, 1] | -- | True |
| │ └─Linear (0) | [32, 128] | [32, 32] | 4,128 | True |
| │ └─ReLU (1) | [32, 32] | [32, 32] | -- | -- |
| │ └─Linear (2) | [32, 32] | [32, 1] | 33 | True |

```
Total params: 6,850,273
Trainable params: 6,850,273
Non-trainable params: 0
Total mult-adds (Units.GIGABYTES): 21.95
Input size (MB): 22.12
Forward/backward pass size (MB): 5569.34
Params size (MB): 27.40
Estimated Total Size (MB): 5618.86
```

همانطور که می توان دید ۶،۸۵۰،۲۷۳ پارامتر قابل آموزش وجود دارد.

## ۱-۲-۳- مرحله پیش آموزش

نخست مرحله پیش آموزش را انجام می دهیم. همانطور که قبلا نیز ذکر شد، در این مرحلـه داده های مجموعه داده APTOS ۲۰۱۵ مورد استفاده قرار می گیرند. ۸۰ درصد این داده ها را به منظور آموزش و ۲۰ درصد باقی مانده را به منظور اعتبارسنجی استفاده کردیم. نکته مهمی که اینجا رعایـت شد این بود که این داده ها به گونه ای انتخاب شدند که دسته ها در داده های آموزش و اعتبارسنجی توزیع احتمالاتی یکسانی داشته باشند.

پیش آموزش به مدت ۱۰ دوره انجام شد. در این مرحله ما از عملیات افزایش داده استفاده ای نکردیم. همه عکس ها بـه وضـوح ۲۴۰×۲۴۰ تنظـیم شـدند. در ایـن جـا از رمزگـذار EfficientNet-B1 استفاده شد. طبق دستورات کد این رمزگذار، میانگین و واریانس مقادیر RGB عکـس هـای ورودی باید به مقادیر زیر تغییر کند:

$$mean = [0.485, 0.456, 0.406] \qquad std = [0.229, 0.224, 0.225]$$

این مقادیر درواقع میانگین و واریانس تصاویر Imagenet هستند که رمزگذار قبلا با این مجموعه داده آموزش دیده است. به همین علت بهتر است که مقادیر پیکسل های تصاویر فوندوس نیز طوری تغییر کنند که میانگین و واریانس آن ها این مقادیر شود.

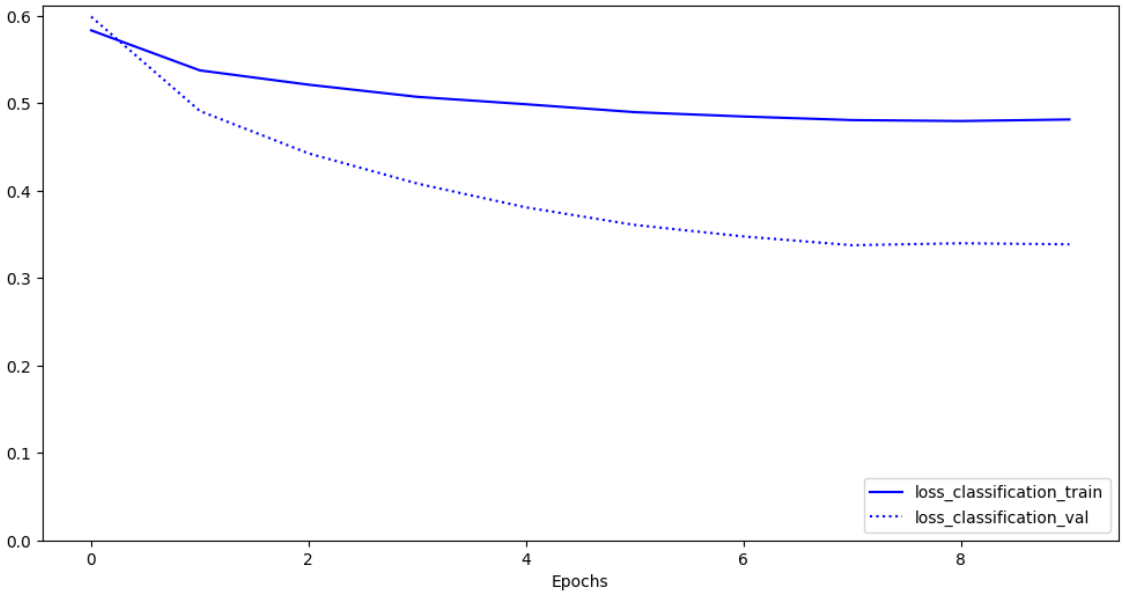
پارامترهای مورد نیاز طبق جدول زیر انتخاب شدند. این پارامتر ها برای دسته بندی باینری و ۵ دسته ای یکسان بودند. برای دسته بندی ۵ دسته ما از هموارسازی برچسـب هـا اسـتفاده کـردیم و مقـدار پارامتر مربوطه آن ۰.۱ انتخاب شد.

| مومنتوم | $\eta_{min}$ | T_Max | کاهش وزن | نرخ یادگیری | دراپ اوت | اندازه بچ |
|---|---|---|---|---|---|---|
| ۰.۹ | ۰.۰۰۰۰۰۱ | ۱۰ | ۰.۰۰۰۰۰۱ | ۰.۰۰۱ | ۰.۴ | ۱۲۸ |

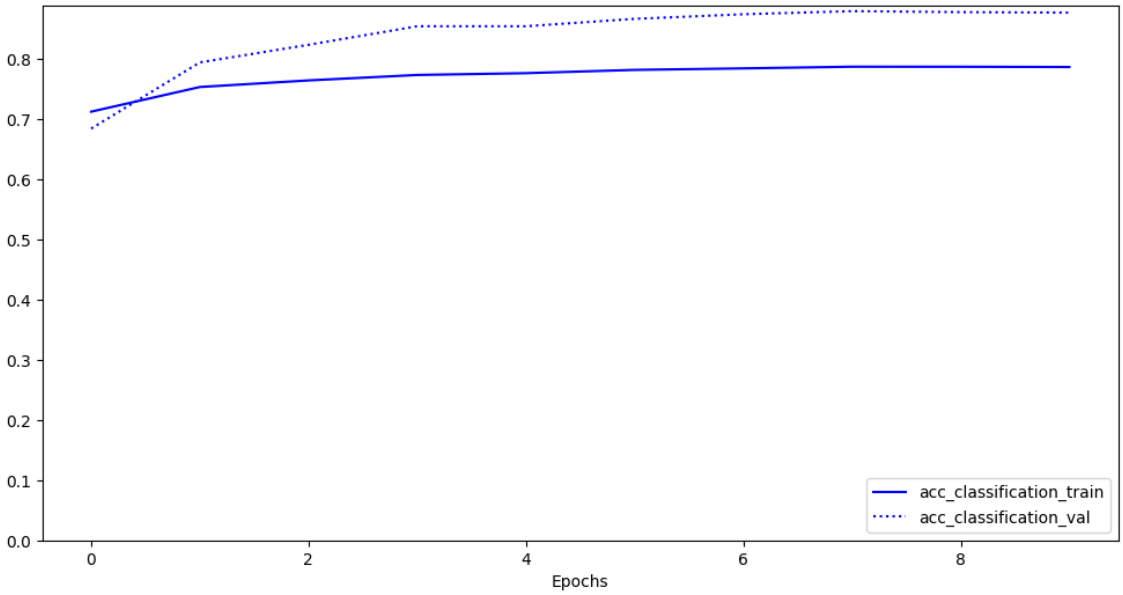
جدول ۱-۳. پارامتر ها برای مدل باینری و ۵ دسته ای

نرخ یادگیری از مقدار ۰.۰۰۱ شروع می شود و به شکل تابع کسینوسی در نهایت به مقدار ۰.۰۰۰۰۰۱ می رسد. زمانی که عدد دوره به مقدار T_Max برسد، نرخ یادگیری به مینیمم مقدار می رسـد. پـس در اینجا نرخ یادگیری افزایش نمی یابد.

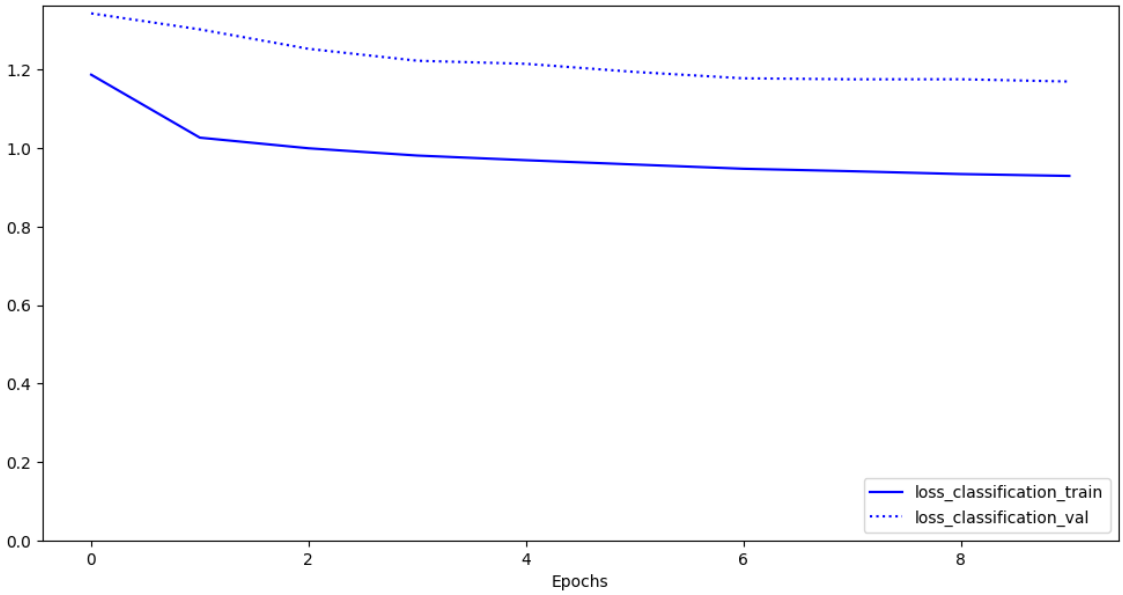
بعد از آموزش مدل، نمودار های خطا و دقت برای داده های مورد استفاده برای آموزش و اعتبارسنجی را رسم کردیم. این نمودار ها کمک قابل توجهی به کنترل فرآیند آموزش می کنند، برای مثال ما باید مراقب باشیم که بیش برازش رخ ندهد و وقتی بیخ برازش رخ می دهد که دقت آموزش افزایش یابد و دقت اعتبارسنجی کاهش یابد. تعداد دوره زیاد می تواند به این پدیده منجر شود.



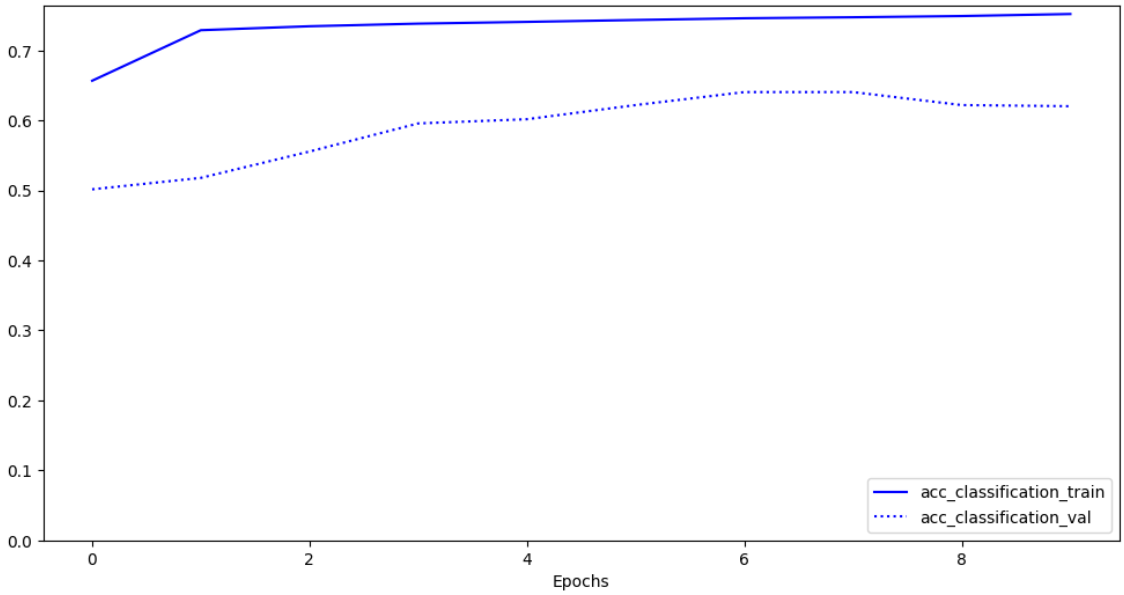

شکل ۱-۳. نمودار خطا برای داده های آموزش و اعتبارسنجی در پیش آموزش مدل باینری




شکل ۲-۳. نمودار دقت برای داده های آموزش و اعتبارسنجی در پیش آموزش مدل باینری




شکل ۳-۳. نمودار خطا برای داده های آموزش و اعتبارسنجی در پیش آموزش مدل ۵ دسته ای




شکل ۴-۳. نمودار دقت برای داده های آموزش و اعتبارسنجی در پیش آموزش مدل ۵ دسته ای

در مدل باینری، در آخرین دوره مقدار دقت اعتبارسنجی به ٪۸۷.۷۱ رسید. در مدل ۵ دسته ای، در آخرین دوره مقدار دقت اعتبارسنجی به ٪۶۲.۰۴ رسید.

وزن ها را در این مرحله ذخیره می کنیم تا در مرحله بعد از آن ها استفاده کنیم.

## ۲-۲-۳- مرحله آموزش اصلی

در این مرحله به آموزش اصلی مدل پرداختیم. همانطور که قبلا نیز ذکر شد، در این مرحله داده های مجموعه داده APTOS ۲۰۱۹، IDRiD و MESSIDOR مورد استفاده قرار می گیرند. اول ۲۰ درصد از داده های APTOS ۲۰۱۹ را به منظور آزمایش نهایی در نظر گرفته و ۸۰ درصد باقی مانده و داده های IDRiD و MESSIDOR را به منظور آموزش و اعتبارسنجی استفاده کردیم، به طوری که نسبت ۸۰ و ۲۰ درصد داشته باشند. باز هم این داده ها به گونه ای انتخاب شدند که دسته ها در داده های آموزش و اعتبارسنجی و آزمایش توزیع احتمالاتی یکسانی داشته باشند.
در این مرحله ما از عملیات افزایش داده استفاده کردیم، به طوری که روی هر تصویر ۳ مورد از عملیات های ممکن به طور تصادفی اجرا شود. همچنین برای مدل ۵ دسته ای پارامتر تمرکز (گاما) عدد ۲ انتخاب شد. همه عکس ها به وضوح ۲۴۰×۲۴۰ تنظیم شدند.
پارامترهای مورد نیاز طبق جدول زیر انتخاب شدند:

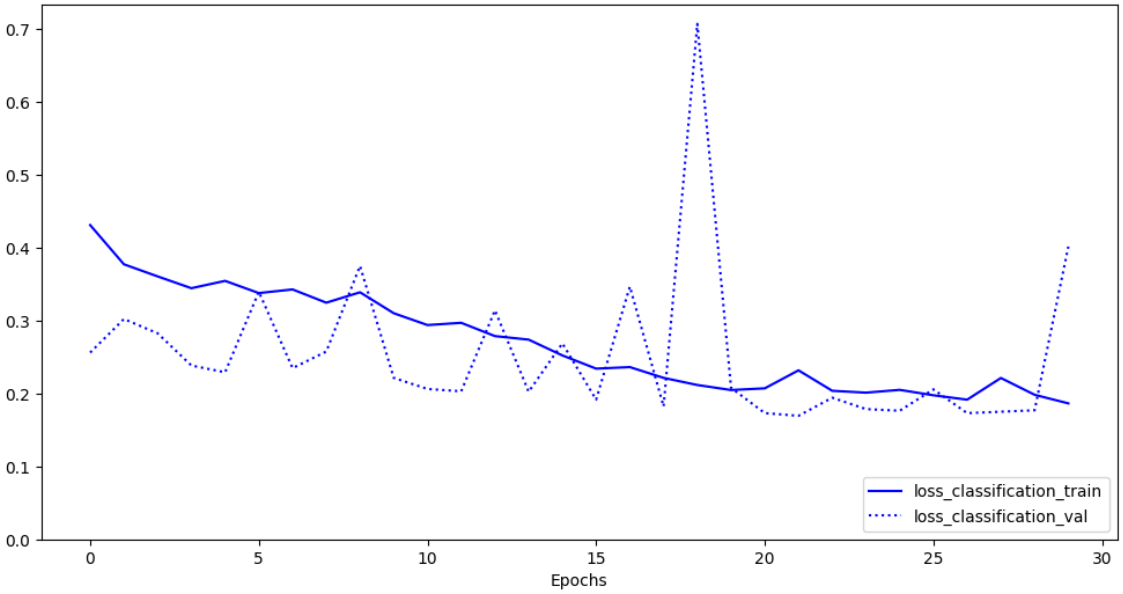
| $\eta_{min}$ | T_Max | کاهش وزن | نرخ یادگیری | دراپ اوت | اندازه بچ | دوره |
|---|---|---|---|---|---|---|
| ۰.۰۰۰۰۱ | ۱۰ | ۰.۰۰۰۱ | ۰.۰۰۱ | ۰.۴ | ۱۲۸ | ۳۰ |

جدول ۲-۳. پارامتر ها برای مدل باینری

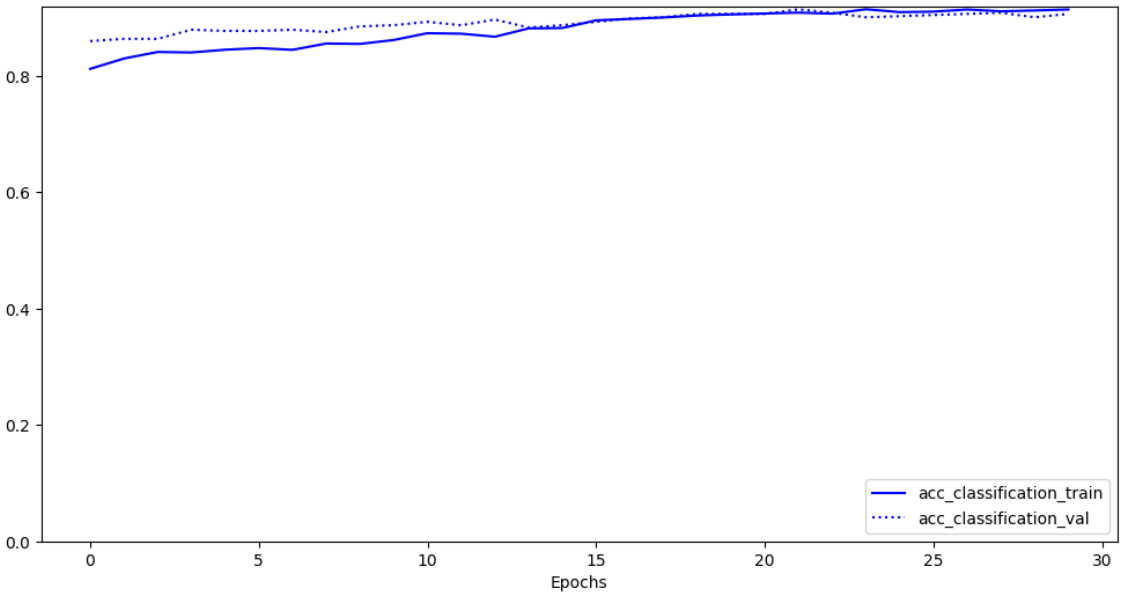
| $\eta_{min}$ | T_Max | کاهش وزن | نرخ یادگیری | دراپ اوت | اندازه بچ | دوره |
|---|---|---|---|---|---|---|
| ۰.۰۰۰۰۱ | ۱۳ | ۰.۰۰۰۱ | ۰.۰۰۱ | ۰.۴ | ۱۲۸ | ۴۰ |

جدول ۳-۳. پارامتر ها برای مدل ۵ دسته ای

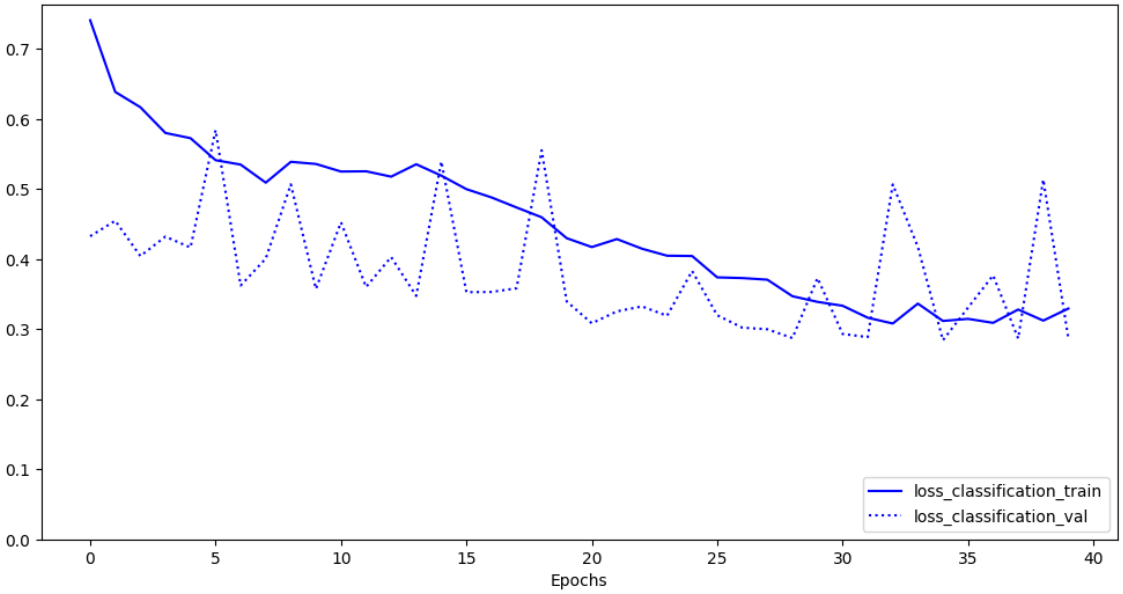
بعد از آموزش اصلی، نمودار های خطا و دقت برای داده های مورد استفاده برای آموزش و اعتبارسنجی را رسم کردیم.



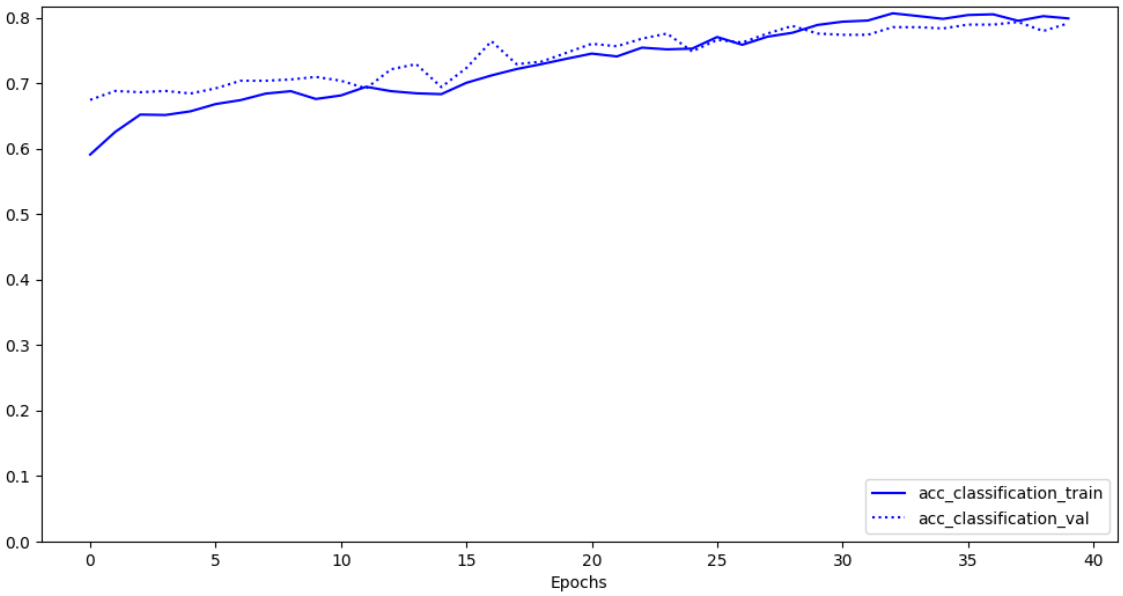

شکل ۵-۳. نمودار خطا برای داده های آموزش و اعتبارسنجی در آموزش اصلی برای مدل باینری




شکل ۶-۳. نمودار دقت برای داده های آموزش و اعتبارسنجی در آموزش اصلی برای مدل باینری




شکل ۷-۳. نمودار خطا برای داده های آموزش و اعتبارسنجی در آموزش اصلی برای مدل ۵ دسته ای




شکل ۸-۳. نمودار دقت برای داده های آموزش و اعتبارسنجی در آموزش اصلی برای مدل ۵ دسته ای

در آخرین دوره، مقدار دقت اعتبارسنجی برای مدل باینری به ۹۰.۶۴٪ و برای مدل ۵ دسته ای به ۷۹.۱۴٪ رسید.

## ۳-۲-۳- نتایج نهایی

در نهایت عملکرد مدل را روی مجموعه داده آزمایش بررسی کردیم.

جدول ۴ و ۵ معیارهای ارزیابی به دست آمده رو مجموعه داده آزمایش نشان می دهد. معیارهای ارزیابی به دست آمده توسط مقاله در جدول ۶ است.

| precision | recall | accuracy | F1-Score | QWK |
|---|---|---|---|---|
| 0.9836 | 0.9836 | 0.9836 | 0.9836 | 0.9672 |

جدول ۴-۳. معیار های ارزیابی برای مدل باینری

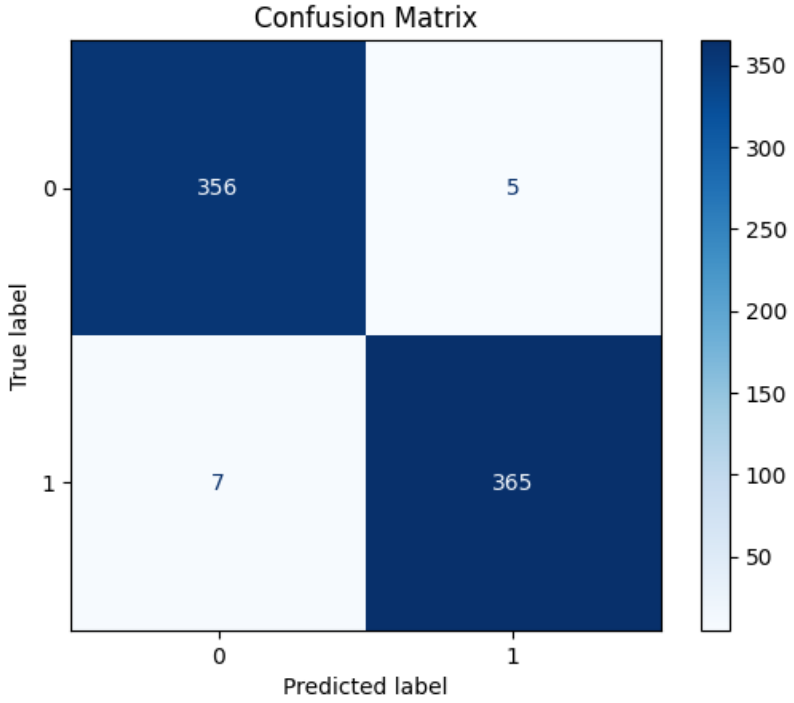
| precision | recall | accuracy | F1-Score | QWK |
|---|---|---|---|---|
| 0.7059 | 0.6533 | 0.8444 | 0.6735 | 0.9054 |

جدول ۵-۳. معیار های ارزیابی برای مدل ۵ دسته ای

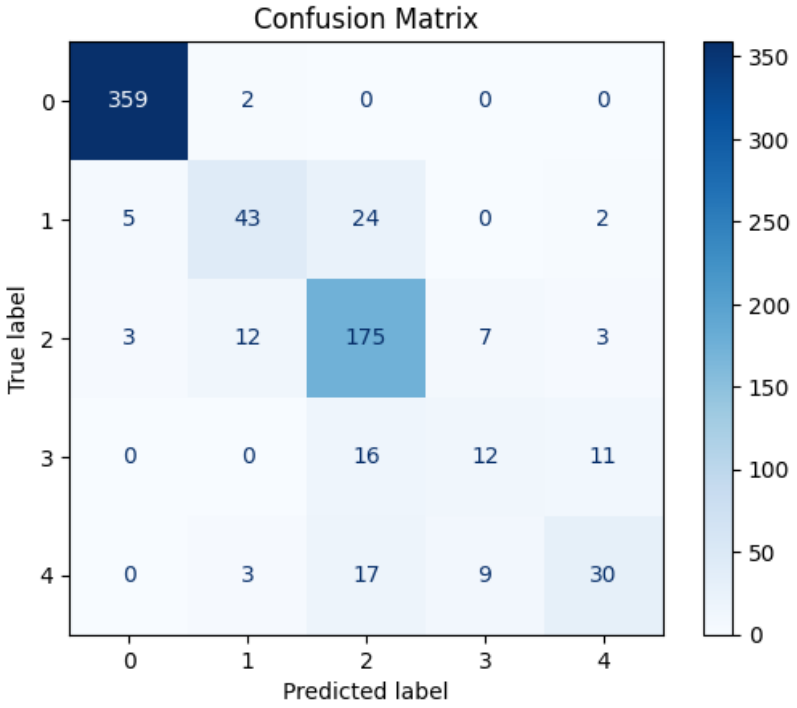
| **Model** | **QWK** | **F1-Score** | **Accuracy** | **Sensitivity** | **Specificity** |
|---|---|---|---|---|---|
| EfficientNet-B4 | 0.965 | 0.811 | 0.903 | 0.812 | 0.976 |
| EfficientNet-B5 | 0.963 | 0.815 | 0.907 | 0.807 | 0.977 |
| SE-ResNeXt50 (512×512) | 0.969 | 0.854 | 0.924 | 0.871 | 0.982 |
| SE-ResNeXt50 (380×380) | 0.960 | 0.788 | 0.892 | 0.785 | 0.974 |
| Ensemble (mean) | 0.968 | 0.840 | 0.921 | 0.8448 | 0.981 |
| Ensemble (trimmed mean) | 0.971 | 0.862 | 0.929 | 0.860 | 0.983 |
| Ensemble (trimmed mean, binary classification) | 0.981 | 0.989 | 0.986 | 0.991 | 0.991 |

جدول ۶-۳. معیار های ارزیابی برای مدل باینری و ۵ دسته ای به دست آماده در مقاله اقتباسی

همچنین ماتریس در هم ریختگی [1]را برای مجموعه داده آزمایش رسم کردیم:



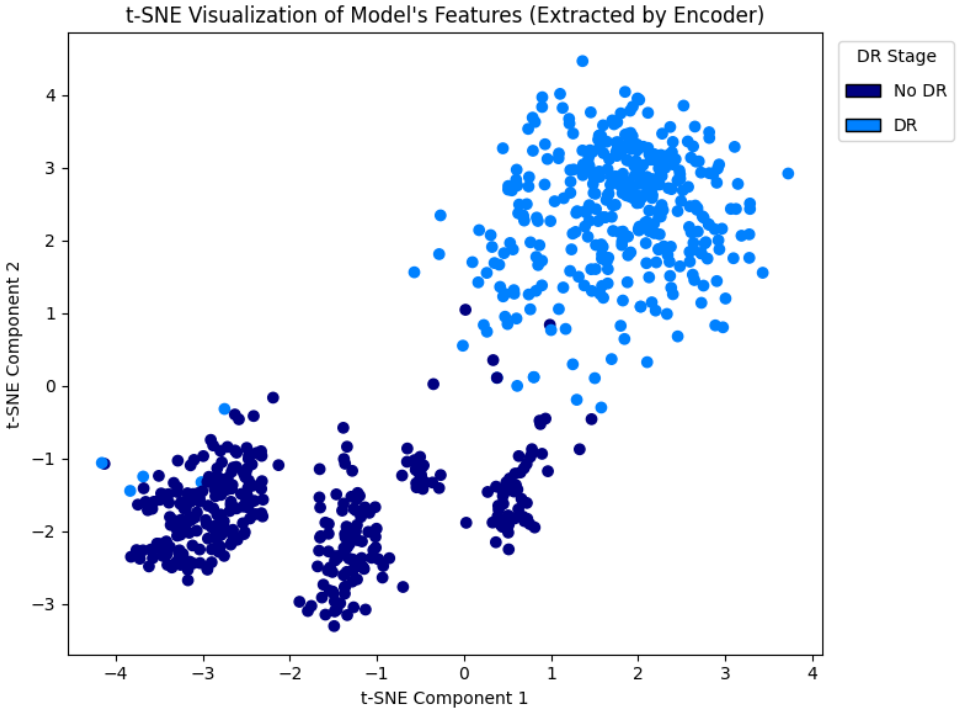

شکل ۹-۳. ماتریس در هم ریختگی برای مجموعه داده آزمایش برای دسته بندی باینری



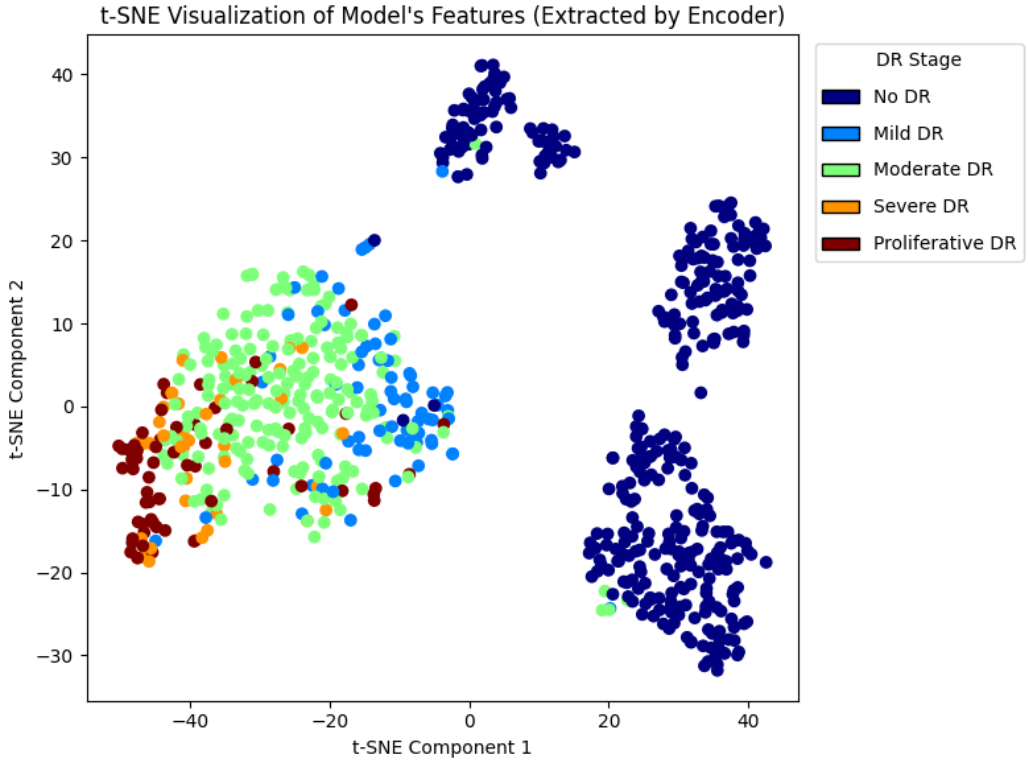

شکل ۱۰-۳. ماتریس در هم ریختگی برای مجموعه داده آزمایش برای دسته بندی ۵ دسته ای

[1] Confusion Matrix

در نهایت نمودار t-SNE را نیز برای مجموعه داده آزمایش رسم کردیم.




شکل ۱۱-۳. نمودار t-SNE برای مجموعه داده آزمایش برای دسته بندی باینری




شکل ۱۲-۳. نمودار t-SNE برای مجموعه داده آزمایش برای دسته بندی ۵ دسته ای

همانطور که در نمودار t-SNE برای مدل باینری می توان دید، بین نمونه های برچسب های ۰ و ۱ فاصله زیادی است و مدل به خوبی توانسته این دو دسته را از هم تفکیک کند. در نمودار t-SNE برای مدل ۵ دسته ای، فاصله نمونه ها با برچسب ۰ بسیار بیشتر از فاصله بین دیگر خوشه ها می باشد. به طور کلی تفکیک بین نمونه ها با برچسب ۱ تا ۴، کاری دشوارتر برای مدل ۵ دسته ای می باشد و به همین علت معیارهای ارزیابی نمرات پایین تری نسبت به مدل باینری دارند.
در نهایت هدف اصلی این پروژه دسته بندی باینری بود که این هدف برآورده شد چرا که که معیار های ارزیابی در مدل باینری نمرات بسیار نزدیکی به نمرات به دست آمده در مقاله اقتباسی داشتند.

## ۳-۳- ابزارها و محیط‌های محاسباتی مورد استفاده

کدهای این پروژه با استفاده از کتابخانه‌ی PyTorch و به صورت مدولار پیاده‌سازی شده‌اند. برای فرایند آموزش مدل‌ها، از کارت‌های گرافیک متعددی بهره گرفته شد. به طور کلی، توسعه‌ی کدها و اجرای آزمایش‌ها در محیط‌های محاسباتی مختلفی انجام شده است؛ از جمله:

- آزمایشگاه کامپیوتر مخابرات، دانشگاه صنعتی شیراز
- زیرساخت ابری ابر فردوسی
- بستر vast.ai به‌عنوان یک سرویس بین‌المللی مبتنی بر لینوکس با کارت‌های گرافیکی قدرتمند

برای آموزش نهایی مدل‌ها، از کارت گرافیک RTX 5080 استفاده شد و نتایج حاصل در مقاله‌ی حاضر گزارش شده‌اند. مدت زمان آموزش مدل در کل حدود ۱ ساعت شد.
پیاده‌سازی پروژه برای مدل دودسته‌ای و مدل پنج‌دسته‌ای در مخزن‌های گیت هاب [۱] زیر در دسترس است:

- **مدل ۲ دسته ای (Binary Model)**
- **مدل ۵ دسته ای (Multi-class Model)**

[۱] GitHub

## ۴-۳- نتیجه گیری

در این فصل، فرایند آموزش مدل تشریح شد. ابتدا مرحله‌ی پیش‌آموزش توضیح داده شد و پس از اتمام آن، نمودارهای مربوط به خطا و دقت برای دو مدل دودسته‌ای و پنج‌دسته‌ای ترسیم گردید. سپس آموزش اصلی مدل آغاز شد و در پایان آن نیز نمودارهای خطا و دقت برای هر دو مدل ارائه شدند. در ادامه، مدل نهایی بر روی مجموعه‌داده‌ی آزمایشی ارزیابی شد.

در مدل دودسته‌ای، نتایج به‌دست‌آمده به شرح زیر است که نشان‌دهنده‌ی بهترین عملکرد مدل می‌باشد:

- Accuracy: 98.36%
- F1-Score: 0.9836
- QWK: 0.9672

همچنین در مدل پنج‌دسته‌ای، نتایج زیر حاصل شد:

- Accuracy: 84.44%
- F1-Score: 0.6735
- QWK: 0.9054

در پایان، نمودارهای ماتریس درهم‌ریختگی و t-SNE برای نمایش فضای ویژگی‌ها ترسیم شدند که عملکرد هر دو مدل را به‌خوبی نشان می‌دهند.

# فصل چهارم: جمع بندی و پیشنهادها

## ۴-۱- پرسش مهم

در این بخش به یک پرسش مهم پاسخ می دهیم: چرا عملکرد مدل باینری بهتر از مدل ۵ دسته ای است؟ به نظر ما این امر به دو دلیل عمده اتفاق می افتد:

۱- عکس های کلاس ۰ بیشترین فراوانی را دارند. در کلاس بندی باینری، توزیع داده ها به صورت ۵۰٪ کلاس ۰ و ۵۰٪ کلاس ۱ شد. این یک توزیع ایده آل است. شبکه عصبی در این حالت بهترین یادگیری را خواهد داشت زیرا از هر دو کلاس به یک اندازه داده مشاهده می کند. نکته دیگری که وجود دارد این است که این داده ها همگی واقعی هستند و داده ها با روش عملیات افزایش داده به این توزیع نرسیده اند. تجربه نشان می دهد که افزایش داده های واقعی، بهترین راه برای بهبود عملکرد شبکه عصبی در پیش بینی است. هیچ کدام از روش هایی مانند عملیات افزایش داده، افزایش داده کلاس های کمیاب تر [۱] یا کاهش داده کلاس های فراوان تر [۲] یا روش های دیگر برای جبران نامیزان بودن دیتاست، به اندازه تهیه داده واقعی بیشتر موثر نیستند.

۲- تفاوت بعضی کلاس ها با یکدیگر کم است. برای مثال تصاویر مربوط به کلاس ۱ مکررا با عکس های مربوط به کلاس ۰ اشتباه گرفته می شوند. این موضوع نه تنها برای مدل، بلکه برای متخصصان انسان نیز صدق می کند. تفاوت های جزئی بین بعضی کلاس ها وجود دارد، که با کاهش رزولوشن عکس این اطلاعات نیز ممکن است از بین بروند. این مسئله مشکلی است که به راحتی قابل حل نیست، چرا که برای کاهش محاسبات و افزایش سرعت آموزش مدل، ما مجبور هستیم که وضوح را پایین بیاوریم.

## ۴-۲- پیشنهادها

یکی از کارهایی که می توانستیم در این پروژه برای تحلیل بهتر انجام دهیم، مطالعات فرسایشی بود. مطالعات فرسایشی یک روش تحلیلی در یادگیری عمیق است که اهمیت هر جزء مدل یا مؤلفه‌های آموزشی را مورد سنجش قرار می‌دهد. در این روش ابتدا مدل کامل آموزش داده شده و عملکرد آن ثبت می‌شود، سپس با حذف یا تغییر هر بخش، اثر آن بر عملکرد مدل بررسی می‌گردد. این مطالعه نشان می‌دهد که هر بخش یا ویژگی تا چه حد در بهبود دقت و کارایی مدل نقش دارد.

در نهایت استفاده از مدل هایی با پارامتر بیشتر مانند EfficientNet-B4 نیز می توانست امتیاز های کسب شده را بهبود دهد.

[۱] oversampling

[۲] undersampling

# مراجع


[1] Rohan T, Frost C, W. N. (1989). Prevention of blindness by screening for diabetic retinopathy: a quantitative assessment.

[2] Krause, J., Gulshan, V., Rahimy, E., Karth, P., Widner, K., Corrado, G. S., Peng, L., and Webster, D. R. (2017). Grader variability and the importance of reference standards for evaluating machine learning models for diabetic retinopathy. CoRR, abs/1710.01711.

[3] **[1]** B. Tymchenko, P. Marchenko, and D. Spodarets, "Deep learning approach to diabetic retinopathy detection," *arXiv preprint arXiv:2003.02261*, Mar. 2020.

[4] Buslaev,A.Parinov,E.K.V. I. I. andKalinin,A.A. (2018).Albumentations: fast and flexible image augmentations.ArXive-prints.

[5] Cheng, J. (2007). A neural network approach to ordinal regression .CoRR,abs/0704.1028.

[6] regression.CoRR,abs/0704.1028.**https://medium.com/data-science/mobilenetv2-inverted-residuals-and-linear-bottlenecks-8a4362f4ffd5**

[7] **https://github.com/FrancescoSaverioZuppichini/BottleNeck-InvertedResidual-FusedMBConv-in-PyTorch**

[8] Loshchilov, I. and Hutter, F. (2016). SGDR: stochastic gradient descent with restarts. CoRR, abs/1608.03983.

[9] **https://docs.pytorch.org/docs/stable/generated/torch.optim.SGD.html**

[10] van der Maaten, L. and Hinton, G. (2008). Visualizing data using t-SNE. Journal of Machine Learning Research, 9:2579–2605

[11] Lin, T., Goyal, P., Girshick, R. B., He, K., and Doll´ar, P. (2017). Focal loss for dense object detection. CoRR, abs/1708.02002.

[12] Tan, M. and Le, Q. V. (2019). Efficientnet: Rethink ing model scaling for convolutional neural networks. cite arxiv:1905.11946Comment: Published in ICML 2019.

## Abstract

Diabetic retinopathy is a diabetes-related eye disease caused by damage to the retinal blood vessels and, if not diagnosed and treated in a timely manner, may lead to blindness. As one of the leading causes of blindness worldwide, its early detection is crucial in preventing the progression of vision-threatening complications. Traditional diagnostic methods often require specialized examinations and are time-consuming; hence, the use of artificial intelligence and deep learning can significantly improve both the accuracy and efficiency of the diagnostic process. In this study, an automated diabetic retinopathy detection system was developed using convolutional neural networks (CNNs) and validated on reliable datasets, including APTOS 2019, APTOS 2015, IDRiD, and MESSIDOR.

The system architecture was inspired by a selected research paper and implemented through transfer learning with the EfficientNet-B1 encoder. Two models were developed: a binary classification model, achieving an accuracy of 0.9836 and a QWK score of 0.9672, and a five-class classification model, achieving an accuracy of 0.8444 and a QWK score of 0.9054.



دانشگاه صنعتی شیراز
SHIRAZ UNIVERSITY
OF TECHNOLOGY

**Department of Electrical Engineering**

**B.Sc. Thesis in**
**Electrical Engineering**

# Classification of Diabetic Retinopathy Using Deep Learning

**By:**
**Amirreza Masoudi**

**Supervisor:**
**Dr. Habibollah Danyali**

**Shahrivar 1404**